AI Debug 不是修不好，而是第一刀常常切錯

Atlas 中文導讀手冊

Problem Map 3.0 Troubleshooting Atlas 是 Problem Map 系列的第三代版本，目前也是 WFGY 主打的核心產品之一。這一版的重點，不只是列出常見問題，而是進一步把 AI Debug 常見故障整理成可路由、可診斷、可延伸修復的地圖系統。這裡的 **Atlas** 一詞，原本就有「地圖冊」的意思；同時，我也刻意借用了先前接近地球的隕石 **3I ATLAS** 這個名字，讓它多一層「正在逼近、值得警覺、需要提早看懂」的象徵感

WFGY(萬法歸一)系列作者：阿紫 BigBig (PSBigBig)
Github：https://github.com/onestardao/WFGY

一份用人類看得懂的方式，重新解釋 AI debug 為什麼常常從錯的地方開始修

第一章

AI 開發越來越快，為什麼 debug 反而更容易走錯路

1.1 這已經不是未來式了，大家真的都在用 AI 寫 code

如果你最近有在寫程式、帶專案、做自動化、搞 agent、碰 RAG，甚至只是默默地在 IDE 裡跟 AI 對話，你大概早就感受到一件事了，AI 寫 code 這件事，已經不是什麼新奇玩具，也不是極客圈的小眾儀式，它已經變成很多人每天真的在用的工作方式。Stack Overflow 2025 開發者調查顯示，84% 的受訪者正在使用或計畫使用 AI 工具在開發流程中，專業開發者裡有 51% 已經是每天使用。GitHub 2024 的跨國調查也顯示，超過 97% 的受訪者曾在工作中使用過 AI coding tools。這個趨勢很清楚，AI 已經不是「會不會進來」的問題，而是「進來之後，真正卡住大家的是哪一段」的問題。

而且，大家會用，不是因為大家集體中邪，而是因為它確實有用。某些任務上，它真的能讓開發變快。GitHub Copilot 的受控研究裡，拿到 AI pair programmer 的實驗組，在指定的 JavaScript 任務上完成速度快了 55.8%。這類結果至少說明一件事，對起手、樣板、局部生成、重複性工作來說，AI 的確能提供非常真實的加速感。這也是為什麼它會迅速滲進開發流程，不是因為大家愛看未來感，而是因為它在很多時候真的省時間。

所以，這一章不是要唱衰 AI。相反地，我們先承認一件事，AI 寫 code 變快，這件事大致是真的。問題是，**寫得快**，跟 **修得對**，從來不是同一件事。這兩件事表面上像堂兄弟，實際上常常住在不同城市，中間還隔著一條名叫「你以為它懂，其實它先切錯地方」的大河。

1.2 速度變快了，但痛點沒有消失，它只是搬家了

很多團隊現在的真實感受，不是「AI 沒幫助」，而是「**AI 很有幫助，但痛點搬家了**」。

以前最痛的地方，可能是第一版寫不出來，API 不知道怎麼接，測試骨架不想手刻，樣板重複到懷疑人生。現在，這些地方常常確實比較快了。AI 會幫你補出

一段像樣的函式、搭出一個初始骨架、把重複性工作變得比較機械化。可是，當專案一進入真實世界，也就是進入長上下文（Long Context）、多檔案（Multi-file）、真實依賴（Real Dependency）、舊 codebase、奇怪 log、模糊症狀、混合故障這些區域，新的瓶頸就慢慢浮上來了。

Stack Overflow 2025 的調查很有意思。最多人抱怨的，不是 AI 完全亂寫，而是它給出「幾乎對，但又不完全對」的答案，這一項高達 66%。接著排在前面的痛點之一，就是「debugging AI-generated code is more time-consuming」，比例是 45.2%。這兩個數字放在一起看，味道就出來了。大家不是單純在抱怨 AI 不會寫，而是在抱怨另一種更煩的情況，那就是它寫得**很像會**，但你要花額外時間把它從「差一點」拉回「真的對」。

這種「差一點」其實非常危險。完全錯的答案，反而比較好處理。你一看就知道不對，直接丟掉，心情雖然不好，但方向很明確。真正麻煩的是那種七分像、八分順、九分自信的答案。它不會立刻被你打槍，甚至還會讓你心裡浮現一個很危險的念頭：「好像只差一點點，我再補一下應該就好。」然後故事就開始了。你補一點，它偏一點。你再補一點，它再偏一點。最後你跟 AI 兩個人一起很努力地把車門烤得金光閃閃，然後才發現，原來一開始沒油。

更麻煩的是，當任務從受控小題變成你自己熟悉的真實專案時，速度優勢也不一定還在。METR 2025 對經驗豐富、而且熟悉自己 repository 的開源開發者做隨機對照研究，結果發現，使用當時前沿的 AI 工具，平均反而花了更久，整體時間增加約 19%。這不表示 AI 毫無價值，而是提醒我們，**真實開發現場的問題，跟受控環境裡的生成題，根本不是同一種運動**。前者不只是在寫東西，還包括判讀、驗證、定位、排除、切層、回溯、確認根因。也就是說，真正難的常常不是「能不能生成」，而是「能不能從正確地方開始修」。

1.3 真正折磨人的，從來不是它完全不會

這裡要講一句很關鍵的話。

真正折磨人的，從來不是 AI 完全不會。真正折磨人的，是它明明很像會，講得也很有道理，卻常常把你帶進一條很完整、很認真、但從第一步就錯掉的修法。

你大概很熟這種感覺。
它不是在胡言亂語。
它不是輸出一坨外星文。
它甚至還會引用你剛剛貼的錯誤訊息，幫你整理成一段條理分明、看起來非常像專業判讀的說明。問題就在這裡，**它太像對了**。

你會看到它很自信地說，問題應該在這裡。
你照做。
結果又冒出新的問題。
它再很自信地說，這次我看到了，真正問題在那裡。
你再照做。
結果系統變得更亂。
這時候最可怕的不是 bug 還沒修好，而是你開始不確定，到底是系統複雜，還是你們兩個已經一起走進了一條結構性錯誤的修路。

這也是為什麼 Atlas 的主張不是「再多發明幾個 bug 名字」，而是先把焦點拉回一個比較殘酷、也比較有用的問題：

你現在修的，真的是最早先壞的地方嗎。

WFGY 的 Atlas 主頁其實把這件事講得很直白。它的核心承諾不是單純幫你命名 failure，而是幫人和 AI 系統更可靠地做五件事，分類 failure、辨認壞掉的規則（Broken Invariant）、把相鄰但容易混淆的失敗區分開、選對第一個修法方向、以及避免後續 debug 崩成 ad hoc 猜測。它甚至把整件事濃縮成一句話，最大的成本常常不是最後答案，而是**第一個修錯的動作**。

說白一點，很多 AI debug 不是修不好，而是第一刀就切錯。後面越修越亂，不一定是因為大家不努力，也不一定是因為模型太笨，而是因為你們兩個從一開始就站錯科別。像看病掛錯科一樣，你不是沒有看醫生，你只是先去了錯的樓層。Atlas 要做的，就是把這個「先去對科別」的能力，從直覺變成比較可重複的系統。

1.4 情境模擬，明明是儀表板和接地出了問題，AI 卻一路把你帶去修引擎

下面這段是**情境模擬**，用來幫助理解 AI 為什麼可能會被上下文（Context）帶偏。它不是在宣稱所有真實 AI 一定都這樣，但很適合拿來理解「越修越錯」是怎麼發生的。

想像一下，你前面整段對話都在聊一台車最近很沒力。你們討論了引擎、動力、傳動、加速、踩油門沒反應、馬力不足，整個上下文裡都是動力系統的詞。你越講越像在處理一台「引擎有問題」的車。

這時候你貼上一段新的故障紀錄。真正重要的資訊其實是這句：

「儀表板油量顯示異常，前一站明明有加油，但油量讀值不穩，而且故障燈有時亮有時不亮。」

如果你是老技師，你可能會先皺一下眉頭。
因為這段訊號不像純引擎故障，它有一股很強的觀測異常（Observability Failure）和接地異常（Grounding Failure）的味道。換句話說，這台車不一定是動力系統先壞，有可能是你先看錯了資訊來源，或根本拿到的是失真的儀表訊號。

但 AI 很可能會發生另一種事。因為前面整段上下文都在強調引擎、傳動、動力，它就很自然地把新訊號吸進原本那條故事線裡。於是它開始很合理地分析火星塞、噴油嘴、節氣門、傳動效率，講得不但很順，甚至還會順便列出幾個你聽起來也覺得不無道理的排查步驟。

問題是，它不是完全亂猜。
它是被上下文裡最濃、最近、最像答案的那一層吸走了。

GitHub Copilot 的官方說明就很適合拿來理解這種現象。GitHub 說，Copilot 在生成建議時，會查看你編輯器裡游標前後的程式碼，也可能利用其他開啟的檔案、repository URL 或 file paths 來抓取相關上下文；在 chat 建議裡，還會把你的 prompt 與 active document、選取內容、workspace 的框架、語言、依賴資訊組合成 contextual prompt。這種工作方式，本來就很擅長利用附近訊號與工作空間線索做下一步預測。我的推論是，這種機制一旦碰到症狀混合、線索濃淡不均、根因被埋在前面或下面的 case，就很容易先抓到「最近、最亮、最像故事主角」的那個東西，而不一定是最早先壞的地方。

這也就是很多人都體驗過的情況。
簡單 bug 時，AI 第一次可能修錯。你再補一段 log，它突然醒來，喔，原來不是這個，是那個。這種 case 還算好，因為你只多繞一圈。
真正麻煩的是複雜 bug。當症狀混在一起，錯誤訊號彼此污染，前後文又長到像一鍋煮太久的火鍋，AI 就更容易先黏到表面症狀，然後很自信地一路往下修。這時候你最需要的，就不是再多一段漂亮 patch，而是一張能幫你先問對問題的地圖。

1.5 為什麼 AI 會第一刀切錯

這件事如果用白話講，其實不神祕。

第一，表面症狀通常比根因更顯眼。

真正的根因（Root Cause）常常藏在比較早的步驟、比較暗的角落、比較無聊的 log、比較不吸睛的格式錯誤，甚至只是某一段看起來不起眼的狀態斷裂。相反地，表面症狀通常比較有戲劇性，很適合被當成故事主角。模型如果太早把故事講順，就很容易把「最像原因的現象」誤認成真正原因。Atlas 主頁也正是從這裡切入，它明講很多看起來像 hallucination 的問題，實際上可能起點是接地漂移(Grounding Drift)；很多看起來像推理崩壞，起點可能是形式容器(Formal Container）先壞；很多像安全或記憶問題的東西，起點可能其實是觀測不足或執行閉合失敗。

第二，上下文不是中立的，它有濃度，有重力。

當某些詞、某些假設、某條故事線在對話裡反覆出現，它們就像實驗室桌上的巨大磁鐵，會把附近的小鐵片全部吸過去。問題是，真正的 bug 不一定住在那塊磁鐵上。你前面聊了十輪「推理怪怪的」，後面真的可能只是資料接錯。你前面一直懷疑「模型忘了」，後面真正的問題也可能是狀態沒接上，不是記憶失憶。AI 類工具的上下文整合能力，對生成來說很強，但對根因定位來說，不保證天然正確。

第三，證據不夠時，人和 AI 都會有一種衝動，想要先完成故事。

這不是 AI 的專利，人類也會。但 AI 的危險之處在於，它完成故事的速度太快，而且口條太好。你還在想「會不會是 A」，它已經幫你寫出一篇三段式分析，順便附修法了。這種能力在生成初稿時是福，在 debugging 的早期分流時就可能是禍。因為 debugging 最怕的不是暫時不知道，而是假裝已經知道。Atlas 之所以把自己定義成 failure router，而不是萬能修復器，就是因為它想先把「假裝已經知道」這件事按住。

第四，簡單問題可以靠新增證據修正，複雜問題則會把錯誤路線放大。

你多貼一段 log、多貼一張 trace、多補一個 before/after，模型有時確實會立刻修正判斷。這說明它不是完全沒救，它只是很容易在證據不足時，先把最顯眼的那個東西當成起點。可是一旦問題夠複雜，表面症狀又彼此牽連，錯的第一判斷就會帶來錯的第一修法，錯的第一修法再生出新的症狀，然後新的症狀又反過來鞏固錯的故事。這時候你看到的就不只是 bug，還會看到一場非常努力的誤會。

1.6 從這裡開始，我不再只問哪裡壞，而是問哪裡先壞

這就是為什麼我後來不想再只是多列幾個 bug 名字。

因為光知道「怪怪的」、「亂掉了」、「不穩」、「在胡說」，其實沒有幫助。
這些都是症狀，不是地圖。
真正有用的問題應該是：

它到底是哪裡先壞。
它現在最像哪一區的 failure。
哪個相鄰區域很像，但其實很可能是誤判。
第一步到底該先修哪裡，不該先修哪裡。

而這，也就是 Atlas 真正想做的事。

它不是在跟你搶修 bug 的光環。
它也不是想假裝自己是一台全自動萬能修復機。
它想做的，是在最容易走錯路的那個瞬間，先幫你和 AI 把第一刀切得比較像樣一點。Atlas 主頁對自己的定義很清楚，它的核心承諾，是幫人與 AI 避免一開始就走進錯的 repair path，先把 failure 活在哪裡、哪一區只是看起來很像、什麼該先修、什麼不該先修，這些事情理清楚。

如果把 AI debug 想成修車，那 Atlas 不是幫你直接把整台車修好。
它比較像那個不會先跟你一起拆引擎的技師。
他會先蹲下來，看地上的油漬、聞一下味道、敲一下儀表板、問你燈是先亮還是先熄，再很冷靜地說一句：

「先別拆這個。你現在修的，很可能不是最早先壞的地方。」

而這句話，很多時候，比再多一個 patch 更值錢。

本章小結

如果只用一句話總結這一章，那就是：

今天的問題，已經不只是 AI 會不會寫。
而是它很會寫，也很會講，但第一刀常常切錯。

而一旦第一刀切錯，後面整段 debug 的成本，就可能一起被拉高。這也是為什麼我後來想做的，不是再多收集幾個 bug 名字，而是把「哪裡先壞」這件事，整理成一張人類看得懂、AI 也用得上的 troubleshooting atlas。

第二章

如果把 AI 當成一台車，七大類其實超好懂

2.1 為什麼我想用修車來講 AI debug

因為大多數人對 AI debug 的痛苦，並不是來自不夠聰明。
而是來自一種很熟悉的煩躁感：

明明壞掉了，
但不知道到底該先看哪裡。

這種感覺，修車的人懂。
看病的人也懂。
你不一定知道引擎、點火、傳動、電系、車架、儀表板的所有細節，但你至少知道一件事：

車子發不動，不代表每一種壞法都一樣。

有時候是沒油。
有時候是電瓶。
有時候是儀表板騙你。
有時候是方向盤歪掉。
有時候是車架先變形。
有時候不是車不會跑，而是你根本看不到它為什麼沒跑。

AI debug 也是一樣。
很多人一看到 AI 出錯，就很容易把所有問題都叫成「亂掉了」、「幻覺」、「推理怪怪的」、「它又在瞎掰」。
這樣講不是完全錯，但也沒有真的幫上忙。
因為這些比較像症狀，不像地圖。

Atlas 的做法，就是反過來不先問「它看起來像什麼症狀」，而是先問：

它比較像哪一種先壞方式。

一旦這個問題問對，後面的修法就會開始像修車。
不是一看到車不動就先拆引擎，
而是先判斷，到底是油、電、儀表、結構、控制、還是根本看錯訊號。
而 Atlas 目前公開的七大家族，就是這樣一張「先壞位置地圖」。

2.2 七大家族先不用背英文，先把感覺抓住

這七大家族，正式名稱分別是：

接地與證據完整性（Grounding & Evidence Integrity）
推理與推進完整性（Reasoning & Progression Integrity）
狀態與連續性完整性（State & Continuity Integrity）
執行與契約完整性（Execution & Contract Integrity）
觀測與可診斷性完整性（Observability & Diagnosability Integrity）
邊界與安全完整性（Boundary & Safety Integrity）
表徵與定位完整性（Representation & Localization Integrity）

如果只看英文，你很容易覺得像在念某種故障魔法咒語。
但別急。
這一章我們先不用那麼學術。
先把它們翻成人類看得懂的樣子。

你可以先把這七類想成下面這七種車況：

第一種，**你拿到的資訊就不對**。
第二種，**零件都在，但運轉順序出問題**。
第三種，**前面和後面接不起來**。
第四種，**理論上該動，但它沒真的動**。
第五種，**它不是先壞在功能，而是你根本看不到它哪裡壞**。
第六種，**該做和不該做的界線亂了**。
第七種，**裡面未必先壞，是外殼、容器、表達結構先歪了**。

這樣一想，整件事就突然比較像人話了。

2.3 接地類（Grounding），像是你一開始就拿錯油、拿錯地圖、看錯油表

先從最常見，也最容易被誤認的開始。

這類在看什麼

接地類（Grounding）看的，是**你一開始接到的資料、證據、外部事實、工作依據，到底穩不穩**。

用車子講

如果你今天加錯油，
或是導航地圖本身就錯，
或是油表顯示不準，
你後面再怎麼認真開，越努力只會越偏。

這不是開車技術的問題。
是你一開始接到世界的方法就歪了。

放到 AI 裡是什麼感覺

AI 這時候常常會看起來很順。
它不是完全胡說。
它是在一個歪掉的地基上，很認真地蓋出一棟看起來還不錯的房子。

所以這類問題的重點，不是先補文筆，不是先改推理，不是先重寫整段答案。
而是先問：

你接到的資料對不對。
你的依據是不是漂了。
你的外部證據是不是接錯了。
你的檢索、引用、事實錨點，到底是不是穩的。

Atlas 的官方 Demo1 其實就特別拿這種 case 出來做示範，重點非常直接，答案很順，不代表接地正確；有些問題第一步該做的是重新接地，不是重寫語氣。

一句白話總結

接地類（Grounding）不是車不會跑。
是你一開始就拿錯了地圖，還一路很有自信地往錯方向開。

2.4 推理類（Reasoning），像是零件都在，但裡面的運轉順序亂了

這類在看什麼

推理類（Reasoning）看的，是**系統裡面的推進順序、因果鏈、邏輯展開，有沒有在對的軌道上**。

用車子講

引擎、齒輪、油路、電路看起來都在，
但裡面運轉的節奏怪怪的。
該先發生的沒先發生，
不該跳步的地方跳步了，
結果整台車不是完全不動，而是動得很詭異。

放到 AI 裡是什麼感覺

AI 在這種情況下常常很像一個急著交報告的高材生。
它不是沒有知識。
它是把步驟偷吃了，
把中間橋樑略過了，
或者把應該慢慢展開的東西，硬壓成一個看似流暢的結論。

這時候如果你把問題誤判成接地類(Grounding)，你可能會一直去補外部資料。
可是真正壞掉的，未必是資料來源，可能是裡面的推進鏈本身。

一句白話總結

推理類（Reasoning）不是沒有零件。
而是零件在錯的節奏裡運轉。

2.5 狀態類（State / Continuity），像是前後接不起來，訊號在半路掉了

這類在看什麼

狀態類（State）看的，是**前一段和下一段之間，狀態有沒有接上，角色有沒有延續，記憶有沒有斷，前後是不是同一台車。**

用車子講

你前面明明已經打火，
後面卻像剛睡醒一樣完全不知道前面發生過什麼。
或者方向盤知道你要左轉，輪胎卻完全沒收到消息。
訊號不是不存在，
是它在途中斷掉了。

放到 AI 裡是什麼感覺

這種問題很常被誤會成「模型忘記了」。
但很多時候，它不是單純忘記。
而是狀態沒有持續、角色污染了、前後任務沒對齊、上下文沒延續成同一條線。

如果你把它誤判成推理類（Reasoning），你就會一直叫它重新推理。
如果你把它誤判成執行類（Execution），你又會一直補指令。
可是真正壞掉的，可能只是那條本來該連起來的狀態鏈斷了。

一句白話總結

狀態類（State）不是不會做。
是它已經不是同一個連續的「它」了。

2.6 執行類（Execution），像是你已經踩油門了，但車子根本沒真的動

這類在看什麼

執行類（Execution）看的，是**命令、流程、橋接、呼叫、觸發、閉合，這些該真的發生的事情，有沒有真的落地。**

用車子講

你方向盤轉了，
排檔排了，
油門踩了，
儀表板看起來也好像很有誠意，
但車根本沒有真的往前。

這不是「想法不好」。
是「動作沒有閉合」。

放到 AI 裡是什麼感覺

很多人會把這類問題罵成「AI 不夠聰明」。
但 Atlas 官方 demo pack 裡對執行類（Execution）給的訊號很清楚，有些 case 不是先壞在推理品質，而是先壞在 readiness、ordering、bridge 或 liveness，也就是流程骨架根本沒有閉合。這時候最該補的不是智商，而是執行閉合（Execution Closure）。

這一類非常像你明明把所有會議都開完了，
結果信根本沒寄出去。
不是大家沒想法，
是最後那一下沒落地。

一句白話總結

執行類（Execution）不是不會想。
是它該發生的動作，沒有真的發生。

2.7 觀測類（Observability），像是車不是先壞在引擎，而是儀表板先死了

這類在看什麼

觀測類（Observability）看的，是**你能不能看見問題、看見路徑、看見故障點、看見中間發生了什麼。**

用車子講

車子可能真的有問題。
但更先壞的，是儀表板。
故障燈不亮，
油量不準，
轉速表亂跳，
你根本看不到哪裡出了事。

這時候你如果急著修引擎，很可能修半天都像在黑暗裡打蚊子。

放到 AI 裡是什麼感覺

這類問題特別容易讓人暴躁。
因為你不是完全沒有 output，
你是有東西，但看不穿。
你不知道它中間怎麼判斷，
不知道哪一段斷掉，
不知道它到底修到哪裡，
更不知道它哪個步驟開始歪。

Atlas 官方的 Demo 2 很漂亮，它特別拿這種 case 出來講：有些系統不是先壞在答案，而是先壞在可診斷性（Diagnosability）。這時候第一步不是急著修答案，而是先把觀測窗打開。

一句白話總結

觀測類（Observability）不是車一定比較糟。
是你根本看不到它到底糟在哪。

2.8 邊界類（Boundary），像是油門、煞車、方向盤該誰管，整台車突然沒共識了

這類在看什麼

邊界類（Boundary）看的，是**系統裡面哪些事情該做、哪些不該做，哪些地方該停、該拒絕、該保守、該限制，這些界線有沒有清楚。**

用車子講

正常車子裡，
油門有油門的工作，
煞車有煞車的工作，
方向盤有方向盤的工作。
如果某一天這三個開始集體失憶，
你就會看到一台在高速公路上進行哲學思辨的車。

「我現在是不是應該停。」
「我是不是可以再衝一下。」
「這條白線算不算建議值。」

這類車，光看就很危險。

放到 AI 裡是什麼感覺

AI 在邊界類（Boundary）出問題時，不一定是內容差。
它常常是界線模糊。
該停的沒停。
該保留的沒保留。
該說不知道的，硬說知道。
該保守的，衝得像剛喝完兩杯濃縮咖啡。

這類問題如果誤判成觀測類(Observability)，你可能會以為只是 logs 不夠。
如果誤判成推理類（Reasoning），你又會一直補推理鏈。
但真正該修的，也許是「界線」本身。

一句白話總結

邊界類（Boundary）不是系統沒反應。
是它不知道哪裡該停，哪裡不該亂衝。

2.9 容器類（Representation / Localization），像是裡面東西不一定壞，是外殼、槽位、孔位先歪了

這類在看什麼

容器類（Representation / Localization）看的，是**資訊被裝進去的方式、呈現的結構、位置、形式容器、符號承載體，到底有沒有先歪掉。**

用車子講

你要裝的是同一組零件，
但如果車架先歪了，
孔位不對，
支架變形，
箱體不合，
你裡面的東西再正常，也會整台車都看起來不對。

這是非常容易被誤判的一類。
因為它表面上很像推理壞了、很像內容壞了、很像結果奇怪。
但實際上，問題可能是「裝它的那個盒子」先壞了。

放到 AI 裡是什麼感覺

這在公式、表格、結構化輸出、多語切換、版面、符號格式、形式描述裡很常見。
你以為是內容推不動，
其實是表徵形式（Representation）或定位方式（Localization）先歪了。
Atlas 的 Demo 4 就特別拿這種 case 來打，它想證明的不是「AI 不會想」，
而是有些問題真正先壞的是形式容器（Formal Container）或描述保真度（Descriptor Fidelity），所以第一修法應該打在容器，而不是直接怪推理。

一句白話總結

容器類（Representation / Localization）不是裡面的引擎先炸。
很多時候，是裝引擎的那個框先歪了。

2.10 七大類真正的差別，不是名字，而是先壞的位置

這裡是這一章最重要的一段。

如果你只把七大家族看成七個標籤，那你很快就會覺得，這是不是只是把 bug 換個比較高級的名字。但如果你把它們看成**七種先壞方式**，事情就完全不一樣了。

接地類（Grounding）先壞的是你接世界的方法。
推理類（Reasoning）先壞的是裡面的推進鏈。
狀態類（State）先壞的是前後連續性。
執行類（Execution）先壞的是動作閉合。
觀測類（Observability）先壞的是你看見故障的能力。
邊界類（Boundary）先壞的是該停、該做、該拒絕的界線。
容器類（Representation / Localization）先壞的是承載內容的形式結構。

而只要先壞的位置不同，
第一步修法就一定不同。

接地類（Grounding）常常先補資料、補錨點、補證據。
推理類（Reasoning）常常先補步驟、補橋、補中間鏈。
狀態類（State）常常先補連續性、補角色、補記憶接點。
執行類（Execution）常常先補流程閉合、補觸發、補 readiness。
觀測類（Observability）常常先補 logs、補 trace、補 visibility。
邊界類（Boundary）常常先補規則、補限制、補 stop condition。
容器類（Representation / Localization）常常先補格式、補載體、補結構孔位。

這也是為什麼 Atlas 一直把「第一刀」看得那麼重。
因為很多時候，真正把事情搞痛苦的，不是 bug 本身有多玄，而是你一開始就修到錯科去了。Atlas 的設計重點，正是讓不同故障家族導向不同的第一修法方向，而不是一股腦把所有 case 都扔進同一個修法桶。

2.11 這一章你不需要全部背起來，你只要先記住一件事

如果你現在還沒把七大家族全部背起來，完全沒關係。
這章最重要的，不是背完七個英文名詞。
最重要的是先讓腦子裡出現這個新觀念：

AI 出錯，不一定是同一種壞法。
而且真正有差的，常常不是「最後回答對不對」，而是「最早先壞的位置在哪裡」。

只要這件事進腦了，
你後面再看七大家族正式母框架、子家族（Subfamily）、子樹（Subtree）、邊界測試（Boundary Test）、壓力測試（Stress Test），整個 Atlas 就會突然不再像一疊冷冰冰的詞，而比較像一張真的能拿來看路的地圖。

本章小結

如果要用一句最簡單的話收這一章，那就是：

七大家族不是七個比較帥的 bug 名字。
它們比較像七種不同的「先壞方式」。

而只要先壞方式不同，
第一步修法就不能亂用同一把扳手。

這也是為什麼接下來第三章，我們不只要說它們「像什麼」，還要正式講清楚：

這七大母框架到底是怎麼切的，為什麼不是亂分，為什麼它真的會影響你怎麼修。

第三章

這不是七個漂亮名字，而是一張真的會影響修法的地圖

3.1 先講白話版結論：這七類不是在分「症狀」，而是在分「先壞的位置」

如果你只是把七大家族看成七種常見 bug 類型，那你很容易低估它。
因為這套 Atlas 真正想切的，不是表面症狀，而是**故障最早開始長出來的位置**。

這個差別非常大。

表面症狀，常常很會騙人。
看起來像在胡說，未必是推理壞了。
看起來像記憶斷了，未必真的是記憶。
看起來像安全或邊界怪怪的，未必先壞在規則。

Atlas 主頁明講，很多看起來像幻覺（Hallucination） 的問題，真正起點可能是接地漂移(Grounding Drift)；很多看起來像推理崩壞(Reasoning Collapse)，真正起點可能是形式容器(Formal Container)先壞；很多像安全或記憶 trouble 的東西，真正起點可能是觀測不足(Observability)或執行閉合失敗(Execution Closure Failure）。也就是說，最會害人的，往往不是 bug 本身，而是你一開始看錯它住在哪一層。

所以這七大家族真正的價值，不是把 failure 換名字，而是把 debugging 這件事，從「看到症狀就開修」改成「先問它到底住在哪個故障區」。Atlas 對自己的定義也很清楚，它不是單純 checklist，而是一套 debugging decision system，目標是幫人和 AI systems 更穩定地完成五件事，分類 failure、找出壞掉的規則（Broken Invariant）、分開容易混淆的相鄰故障區、選對第一個修法方向、以及避免後面整段 debugging 崩成 ad hoc 猜測。

如果用比較不正經但很準的講法，這七類不是在分「這台車看起來哪裡醜」，而是在分「這台車到底是先哪裡出事」。一台車發不動，你可以描述它很吵、很晃、很怪、很沒力。但真正有用的問題是：

它是沒油。
還是儀表板壞了。
還是車架歪了。
還是油門踩下去根本沒真的傳到輪胎。

七大家族做的，就是把這個「先壞位置」正式固定下來。

3.2 這七大母框架，不是為了好看才切成七類

這裡先講最重要的判斷。

Atlas 目前公開版本，把整個 failure space 組織成七個最上層家族，這不是臨時靈感，也不是先決定要有七種，再把案例硬塞進去。主頁 FAQ 很直接地說，這個母框架不是先選了一個「七聽起來很優雅」的數字，再往裡面塞東西；相反地，它是「在壓力下被刻出來的」。官方原文甚至直接講，重要的不是「七很優雅」，而是到目前為止，在已測試的覆蓋面裡，頂層家族依然可讀、主要邊界切口依然穩定、後續精煉壓力主要落在子樹（Subtree）與節點結構（Node Structure），而不是已經出現必須把母表重畫成第八家族的壓力。

這句話非常關鍵。

因為它代表一件事，現在的七大家族，不是「宇宙終極完美數字」，而是**到目前為止最穩定、最能承載真實 case、邊界也最站得住的母結構**。這其實比「我們已經找到宇宙真理」更可信，因為它承認自己是在真實壓力中被逼出來的，而不是在白板上用喜歡的數字拼出來的。

Atlas 的核心凍結文件也把這件事釘得很死。Atlas Final Freeze v1 明講，第一正式版本現在已經穩定到可以凍結的部分，包括七大家族母表（Seven-family Mother Table）、核心路由規則（Core Routing Rules）、標準節點層（Canonical Node Layer）、家族入口層（Family-entry Layer）、高價值子樹層（High-value Subtree Layer）、關係矩陣（Relation Matrix v1）、第一層 repair-facing mapping，以及 patch protocol。它甚至特別說，這份文件叫 Final Freeze，不是因為所有工作都做完了，而是因為第一正式 Atlas 主體已經穩定到可以被當成可重用的核心版本。未來工作應該走 patch 模式，而不是默默重寫核心。

這個味道就很不一樣了。
它不是在說「看我切了七類，很酷」。
它是在說：

我現在已經有一張可以凍結的母地圖，後面的成長，不該再回頭把地基偷偷拆掉。

這種說法，其實就是可信度。

3.3 七大母框架到底在切什麼

下面我們把它稍微講正式一點，但還是盡量保持人話。

接地與證據完整性（Grounding & Evidence Integrity）

這一類切的是：
系統有沒有穩穩接在外部證據、真實世界錨點（World Anchor）、事實依據、語義目標上。凍結文件裡給它的核心 broken invariant 很清楚，就是 evidence / truth / world-anchor coupling broken。也就是說，真正先壞的，是你接世界的那條線。

推理與推進完整性（Reasoning & Progression Integrity）

這一類切的是：
不是資料來源歪了，而是內部推進鏈、推導順序、展開節奏、橋接步驟出了問題。你可以把它想成零件還在，但運轉節奏不對。這類 case 常常看起來很聰明，卻在中間偷步、跳步、斷橋。Atlas 主頁也特別把它和「看起來像推理崩壞，其實先壞在容器」這種相鄰誤判分開處理。

狀態與連續性完整性（State & Continuity Integrity）

這一類切的是：
前後狀態是不是同一條線，角色是不是延續，前一步和後一步有沒有真的接上。它很像車上的訊號本來該一路傳下去，結果半路掉了。你以為它失憶，其實可能是 continuity 斷了。凍結文件把這一層當成母表的一部分，就是在承認這種 failure 不該被粗暴地塞回推理或執行裡。

執行與契約完整性（Execution & Contract Integrity）

這一類切的是：
該被觸發的流程、該發生的動作、該閉合的橋、該遵守的呼叫契約，有沒有真的落地。這不是「它不會想」，而是「它沒有真的做完」。官方 demo 也直接把這種 case 拿來說明，有些系統先壞的是 execution closure，不是 reasoning quality。

觀測與可診斷性完整性（Observability & Diagnosability Integrity）

這一類切的是：
你到底看不看得見故障。
不是每次都是功能先壞，有時候是儀表板先死。
Atlas 主頁與 demo 介紹都很強調，很多 debugging 的成本不是修不了，而是根本看不見哪裡開始歪。這一類先補的常常不是答案，而是 visibility。

邊界與安全完整性（Boundary & Safety Integrity）

這一類切的是：
哪些該做、哪些不該做、哪些該停、該保守、該拒絕，這些界線有沒有穩。
它不是單純「比較膽小」或「比較保守」，而是系統的 stop condition、rule envelope、limit behavior 到底有沒有守住。這類如果切錯，很容易把邊界誤判成觀測問題，或把安全誤判成推理問題。

表徵與定位完整性（Representation & Localization Integrity）

這一類切的是：
不是裡面的知識先壞，而是承載知識的形式、容器、定位槽位、結構載體先歪。
這一類極容易被誤判。因為表面上很像 reasoning fail，實際上可能是 symbolic representation fidelity、logic descriptor fidelity、formal container adequacy 這一層先壞。主頁與 freeze 文件都把這類放在很核心的位置，因為它正是很多人第一眼最容易看錯的故障區之一。

如果把這七類放在一起看，你會發現它們根本不是「同一種 bug 的七種別名」。
它們切的是七種完全不同的故障起點。這就是為什麼第一修法會跟著變。

3.4 為什麼這七類會真的影響修法，而不是只是比較好看的分類學

這裡要講 Atlas 最值錢的地方。

如果一套分類只是讓你更會命名，那它最多只能讓會議比較好開，簡報比較漂亮。
真正有價值的分類，必須改變你的下一步動作。
Atlas 主頁與 Router Freeze Note 都把這件事講得很清楚，這套系統的重點不是停在 classify，而是要讓模型更有機會回答：

這是哪一種 failure。
為什麼它屬於這裡。
為什麼不是最誘人的相鄰家族。
它壞掉的規則是什麼。
第一步應該往哪個修法方向走。

這裡你就能看出 Atlas 和一般「錯誤清單」的差別。

一般清單比較像：
你現在像哪一種病。

Atlas 比較像：
你現在該先掛哪一科，
而且先不要跑錯樓層。

這個差別非常大。
因為很多 debugging 成本，不是花在最後那一下 patch，而是花在前面那幾次誤修（Misrepair）。Atlas 主頁本身就把 wrong first cut、misrepair、追錯相鄰故障區、太晚看見 broken invariant、以及觀測不足，視為大量時間浪費的來源。

如果你把接地類（Grounding）當成推理類（Reasoning），你可能會一直補思維鏈，結果越補越歪。如果你把容器類（Representation / Localization）當成推理類（Reasoning），你可能會怪模型不會想，卻忽略是裝它的形式先變形。如果你把觀測類（Observability）當成執行類（Execution），你可能會一直補流程，卻根本沒先打開儀表板。

這些不是語意上的小差別，它們會直接決定你第一把扳手往哪裡下。Atlas 的一句核心話可以把整件事講完：

Wrong first diagnosis creates wrong first repair.
「錯誤的初步診斷，會導致錯誤的初步維修。」

這句話很像老派維修師傅會說的東西。
只是 Atlas 把它做成一張正式地圖。

3.5 最容易切錯的，不是家族內部，而是家族邊界

如果你有做過稍微複雜一點的 debugging，你就知道真正危險的地方，往往不是那種一眼就知道的 case。真正危險的，是**長得很像別人的** case。

Atlas 很早就承認這件事，所以它不是只給家族名稱，還特別強調「相鄰又誘人的錯區」這種東西。主頁寫得很清楚，這套系統不是只問 what went wrong，而是還要問 what neighboring region is tempting but wrong，what should be repaired first，what should not be repaired first（哪些相鄰區域看似可疑

卻是錯誤方向？哪些應該優先修理？而哪些又不該先行維修？)。Router Freeze Note 也把這個精神凍結進 quick-map 與 output contract 裡，要求模型不只給 family cut，還要交代 boundary pressure、broken invariant、first repair direction，甚至要說明為什麼它不是最誘人的相鄰家族。

這裡最經典的幾組邊界，就是：

接地類（Grounding） vs 表徵與定位類（Representation / Localization）

表面上都可能呈現成「答案怪怪的」。
但一個是接世界的那條線歪了，
一個是裝答案的盒子先歪了。
你如果把前者誤判成後者，就會一直補格式；
把後者誤判成前者，就會一直補資料。
兩邊都很辛苦，而且都很容易一邊修一邊懷疑人生。

推理類（Reasoning） vs 表徵與定位類（Representation / Localization）

這組超容易吵起來。
看起來都像「它推不動」。
但其中一種是推進鏈本身斷橋，
另一種是符號容器、描述保真度、形式承載體先壞。
也就是說，一個是引擎運轉邏輯亂了，另一個是引擎艙本身先歪了。

狀態與連續性類（State / Continuity） vs 執行與契約類（Execution / Contract）

這組也很常混。
前者是訊號沒接上，
後者是動作沒落地。
前者比較像接力棒中途掉地上，
後者比較像棒子傳到了，但下一個人根本沒起跑。

觀測類（Observability） vs 邊界類（Boundary）

這組更妙。
有時候你以為是規則失守，
其實只是你根本看不到它何時開始偏；

有時候你以為只是 logs 不夠，
其實真正先壞的是它不知道哪裡該停。

這也是為什麼 Atlas 要把「邊界測試（Boundary Test）」當成正式思考的一部分。**因為真正的高手，不是只會分家族，而是知道哪兩家最容易長得像雙胞胎。**

3.6 這套東西不是只停在「分完就好」，而是從 route (路由)走到 first repair(第一次修正)

這裡也很重要，因為很多人看到分類圖，就會直覺地問：

好，分完了，然後呢。

Atlas 的回答不是「分完就散會」。
主頁明講，這個系統不是 classify and stop，而是 route、cut correctly、repair the right layer first**(釐清路徑、準確切割、先從正確的層面修起)**，只有在需要時才往更深的 WFGY 層升級。Fixes 層也把自己定義得很清楚，它是 Atlas 的 repair-facing extension layer，Atlas 負責告訴你 failure lives 在哪裡，Casebook 教你怎麼認那道切口，Adapter 幫 AI 在紀律下做 route，而 Fixes 層開始回答下一題：

正確路由之後，第一步該試什麼。

這也就是為什麼這一章要一直強調「真的會影響修法」。
因為如果你用錯了母框架，後面的 first repair direction 幾乎必然跟著錯。
而如果你一開始切對，哪怕後面還沒全修完，整段 debugging 至少開始進入對的樓層。

這種感覺很像什麼呢。
很像你終於不再對著車門補土、拋光、打蠟，
而是有人蹲下來指著地板說：

「先不要摸那個。你現在真正該先看的，是油路，不是烤漆。」

這個瞬間，地圖的價值就出來了。

3.7 所以，第三章真正想說的只有一件事

如果要把這一章壓成一句話，我會這樣講：

這七大家族不是七個比較帥的名稱。
它們是七種不同的故障起點，而不同的故障起點，會把你帶向不同的第一修法。

這就是為什麼它值得被叫做 Atlas（中文翻譯：地圖冊）
不是因為名字有宇宙感。
而是因為它真的在**做「定位」這件事**。

它不是幫你把 bug 包裝得更文藝。
它是在幫你決定，你現在到底該往哪個方向動手，哪些相鄰故障區雖然很誘人，但很可能是錯的，哪些東西現在不該先修。Atlas 主頁對自己的一句濃縮版本其實已經講完了，Problem Map 3.0 Troubleshooting Atlas，是一套給複雜 AI failures 用的 debugging decision system，目標是減少 wrong-first-fix debugging。

而接下來，第四章就會往下一層走。
也就是讀者一定會追問的那一題：

如果七大家族只是母框架，那下面到底有沒有更細的結構。
還是其實只是七個大標題撐場面。

答案當然不是後者。
而且，這也正是第四章要開始講的地方。

本章小結

這一章做的事情，是把七大家族從「好像有道理」提升成「原來它真的會改變修法」。

重點不是你要把七個英文名詞全部背起來。
重點是你要開始懂：

Atlas 不是在分表面症狀。
Atlas 在分的是哪裡先壞。

而只要先壞的位置不同，
第一步修法就不能亂共用同一把扳手。

第四章

七大類下面，為什麼還需要子家族

4.1 七大類不夠嗎

先講一個最直白的答案。

夠，也不夠。

夠，**是因為七大家族已經足夠讓你在第一眼不要走進完全錯的樓層**。
你至少開始知道，這是**接地與證據完整性**（Grounding & Evidence Integrity）的問題，還是**表徵與定位完整性**（Representation & Localization Integrity）的問題；是**狀態與連續性完整性**（State & Continuity Integrity）先壞，還是**執行與契約完整性**（Execution & Contract Integrity）先壞。光是這件事，已經能幫很多 case 省掉非常多白工。Atlas 主頁和核心凍結文件都把這一層當成正式穩定骨架，因為七大家族母表、家族入口層、高價值子樹層、關係層和第一層 repair-facing mapping 已經被凍結成第一版正式核心。

但不夠，是因為真實世界不是只有「大概哪一區」就能收工。
很多時候你已經知道自己在接地類（Grounding）這一區，問題卻還是很多。
它是**檢索錨點漂移（Retrieval Anchor Drift）**，還是**語義接地錯位（Semantic Grounding Mismatch）**。你知道自己在執行類（Execution）這一區，可是真正先壞的是**啟動排序失敗（Bootstrap Ordering Failure）**，還是**橋接完整性失敗（Bridge Integrity Failure）**。

你知道自己在表徵與定位類（Representation / Localization），但到底是 **OCR／版面結構失敗（OCR / Layout Structural Failure）**，還是**表徵載體扭曲（Representation Carrier Distortion）**。
這些時候，如果沒有更細的層，你就會進入一種很尷尬的狀態：

你知道大概在這附近，
但你還是不知道第一把小螺絲起子要往哪裡插。

所以七大家族像是地圖上的大洲。
夠你知道自己現在在亞洲，不在南美洲。
但如果你接下來真的要走路，還是得有城市、道路、區塊，甚至路口。
子家族（Subfamily） 和 **子樹**（Subtree），做的就是這件事。

4.2 子家族的任務，不是增加複雜度，而是減少誤修

這句要先寫清楚，因為它是這一章的靈魂。

很多人一看到更細的分支，就會下意識覺得：

「喔，又要把事情搞複雜了。」
「又來了，分類學家最愛開枝散葉。」
「你是不是只是想把一個 bug 拆成更多名字。」

但 Atlas 下面的子家族，不該用這個角度看。
它們的任務不是把東西變得更花，而是把**誤修（Misrepair）**變少。

因為誤修的本質，常常不是你完全在錯的世界。
很多時候你已經進到對的樓層了，但你還是在錯的房間外面徘徊。
你知道是接地類（Grounding），卻把資料錨點問題當成目標對齊問題。
你知道是狀態類（State），卻把角色污染看成記憶衰退。
你知道是觀測類（Observability），卻把「根本還沒看見路徑」硬拗成「只是安全規則太保守」。

這種時候，七大家族已經幫你避開大災難，但還不夠幫你下準第一刀。
而子家族的價值，就是在這裡把「差一點就修對」和「繼續修歪」拉開。Router TXT 的核心規則寫得很直接，它不只要求 primary family、secondary family、broken invariant、first fix direction(核心關聯群、次要關聯群、失效的約束條件、第一修復路徑)，還要求 best current fit 必須誠實，能停在家族層（family-level）就停在家族層，只有在更細節點真的有直接證據時，才往節點層（node-level）或未解子類型（unresolved_subtype）走。它甚至明文禁止「主題相似就硬跳到 node-level」，也禁止在證據很薄的情況下演戲式精準。這整套規則的本質，其實就是在防止子家族變成花招。

所以子家族存在的理由，不是「讓地圖看起來更專業」。
而是很現實的一句話：

同一層樓，也可能有很多會把你帶去不同修法的房間。

4.3 Atlas 下面，現在已經不是空白了

這裡先把話講準。

如果你問：「Atlas repo 現在有沒有正式承認下面還有更細的結構。」
答案是，有，而且不是暗示，是直接寫出來。

Atlas Final Freeze v1 明白寫到，第一版正式凍結的東西裡，不只有七大家族母表，還包括**家族入口層（Family-entry Layer）**、**高價值子樹層（High-value Subtree Layer）**、**標準節點層（Canonical Node Layer）**、以及關係矩陣（Relation Matrix）。同一份文件也同時講清楚，這不代表所有子樹都已經完全展開，因為某些子樹還在「持續加厚（Ongoing Subtree Thickening）」的狀態，也就是說，它們已經夠強到值得存在，但仍然受益於更多壓力與案例證據。這個說法其實很成熟，它不是說「全部做完了」，而是說「下面的骨頭已經長出來了，只是有些肌肉還在練」。

而你上傳的 Router TXT，則把這件事再往前推了一步。
它不只承認更細的結構存在，還正式做了一個**適配候選註冊表（Fit Candidate Registry）**。這個註冊表不是亂列名詞，它把每一大類底下更細的候選分支切成幾種層級：

穩定到夠用的（stable_enough）
可以用，但要保守的（use_with_caution）
證據薄時只能停在家族層的（family_level_only_if_thin）
以及不該亂衝的（do_not_overreach）

這個設計非常重要。
因為它代表 Atlas 沒有走那種「既然有子家族，就把所有細節都硬命名」的路。
它反而更像一個有點龜毛的故障植物學家，手上拿著放大鏡說：

「這棵我們確定是橡樹。」
「這棵看起來也像橡樹，但先不要硬說是第幾號變種，證據還薄。」
「那棵不要亂命名，現在只是長得很像。」

這種態度，反而讓子家族更可信。

4.4 我們怎麼知道這些子家族不是亂切的

這一節，就是你最在意的地方。
而且我同意，這一節一定要講，因為不講，外面的人很容易把整套 Atlas 誤讀成「看起來很完整的命名藝術」。

先講結論：

不是亂切。
因為切完之後，還有邊界測試（Boundary Test）、壓力測試（Stress Test）、解析度紀律(Resolution Discipline)、以及不准偷懶審核(No Laziness Audit)在後面盯著。

這不是我幫你美化。
Router TXT 裡真的有這些東西。

它先在結構上要求，七大家族每一類都必須有一套完整的操作化定義，也就是：

什麼先壞（what fails first）
壞掉的規則是什麼（broken invariant）
這一類不是什麼（what this family is not）
最容易混淆的是誰（confusion family）
邊界觸發條件（boundary trigger）
第一修法模式（first fix pattern）
以及最容易發生的誤修陷阱（misrepair trap）

光這一層，就已經不是隨便命名。
因為你只要真的寫過 case，你就知道，很多漂亮分類一碰到「這類不是什麼」就破功。真正站得住的類型，除了會說「我是誰」，還要會說「我不是誰」。

再來，TXT 明確要求幾組邊界必須被 decisionize(**拍板定案第一修復方向**)，也就是不能只說「這兩個很像」，而是要能回答「什麼情況選 A，什麼情況選 B，什麼時候不要太早選 A，什麼時候不要太早選 B，證據門檻在哪裡，平手時怎麼退回粗顆粒判斷」。被點名要求的邊界包含：

接地類（Grounding） vs 表徵與定位類（Representation / Localization）
觀測類（Observability） vs 邊界類（Boundary）
狀態類（State） vs 執行類（Execution）
推理類（Reasoning） vs 表徵與定位類（Representation / Localization）
以及其他幾組相鄰高壓邊界。

還不只這樣。
TXT 還正式列出一組**壓力測試（Pressure Tests）**，專門拿來測這套路由是不是會在關鍵地方塌掉。比如：

能不能分出外部錨點流失（External Anchor Loss）和結構載體失敗（Structural Carrier Failure），不要一股腦把兩者都塞進接地類或容器類。
能不能在觀測不足時，避免太早把 case 丟進邊界類。
能不能分出角色／所有權持續漂移（Role / Ownership Persistence Drift）和排序／橋接／閉合失敗（Ordering / Readiness / Bridge Closure Failure）。
能不能在形式看起來很正式時，仍然承認真正先壞的可能其實是接地，而不是容器。以及，**能不能在證據很薄的時候，老老實實停在家族層，不要假裝節點層很清楚。**

最後，它甚至還有一個非常討喜的東西，叫做**不准偷懶審核（No Laziness Audit）**。
這個區塊直接列出哪些偷懶行為是不准的，例如：

看到 safety 相關字眼就自動丟進邊界類（Boundary）
看到 OCR、schema 之類字眼就自動丟進表徵與定位類（Representation / Localization）
看到 memory 就自動丟進狀態類（State）
看到 execution 相關字就自動丟進執行類（Execution）
看到抽象字眼就自動丟進觀測類（Observability）
以及用浮誇的 secondary family、裝懂的 node-level、假裝精準、把 no-fit 當逃生門、把 first fix 漂成 cosmetic cleanup，這些全部都被列為 fail condition。

老實說，這一段很有靈魂。
因為它等於公開承認了一件事：

真正的敵人，不只是分類失敗。
還有那種「看起來很像在分類，其實只是在偷懶湊答案」的偽精準。

所以子家族如果還能在這些規則下活下來，它就比較不是亂切。

4.5 代表性的子家族長什麼樣，先看幾個就夠

這裡我不會把整棵樹整包丟出來。
那樣會讓這一章變成植物圖鑑，不是導讀手冊。
我們只看幾組代表性的例子，讓你感受到「下面真的有東西」。

接地與證據類（Grounding）下面的代表子家族

Router TXT 把接地類底下穩定度較高的候選列得很具體，例如：

檢索錨點漂移（Retrieval Anchor Drift）
語義接地錯位（Semantic Grounding Mismatch）
合成真值接地（Synthetic Truth Grounding）
分布外世界接地失敗（OOD World Grounding Failure）
訓練到部署的分布接地落差（Train-Deploy Distribution Grounding Gap）

你一看就知道，這不是單純「資料有沒有對」那麼粗。
它已經開始分辨，你到底是檢索錨點漂了，還是語義對齊歪了，還是你在訓練世界裡學到的東西，一到真實部署世界就找不到落地點。

執行與契約類（Execution）下面的代表子家族

這一類底下，TXT 也列出了幾個很有工程味的候選：

啟動排序失敗（Bootstrap Ordering Failure）
部署死鎖（Deployment Deadlock）
未準備好就執行（Pre-Readiness Execution Failure）
橋接完整性失敗（Bridge Integrity Failure）
跨層活性退化（Cross-Layer Liveness Degradation）

這就很像真正做系統的人會碰到的故障，不再只是「它沒做完」。
你開始能看見，那個「沒做完」裡面到底是排序、橋接、活性、還是 readiness 先壞。

觀測與可診斷類（Observability）下面的代表子家族

這一類的候選很有味道，像是：

故障路徑不透明（Failure Path Opacity）
監督覆蓋失敗（Oversight Coverage Failure）
一致性可觀測缺口（Coherence Observability Deficit）
意義輪廓可稽核性失敗（Meaning-Profile Auditability Failure）

光看名字就知道，它不是在講功能壞沒壞，而是在講「你有沒有看見它怎麼壞」。

表徵與定位類（Representation / Localization）下面的代表子家族

這一類底下也已經不是空白，例如：

OCR／版面結構失敗（OCR / Layout Structural Failure）
表徵載體扭曲（Representation Carrier Distortion）
語義定位失敗（Semantic Localization Failure）
錨點與容器錯配（Anchor-Container Mismatch）

這些名稱的厲害之處，不在於術語感，而在於它們真的對應到很不同的故障畫面。你遇到的是版面崩掉、載體扭曲、還是錨點和容器彼此打架，修法當然不可能同一套。

所以，這一章真正要你看懂的不是「好多名字」。
而是：

Atlas 下面不是一片平地。
它已經長出道路、叉路、區塊，甚至路口。

4.6 但我們為什麼不在這份 PDF 裡把整棵樹一次塞滿

因為這份 PDF 的任務，**不是把讀者活埋**。

說真的，很多技術文件會翻車，不是因為作者不夠懂，而是因為作者太想一次證明自己很懂。於是他把所有分支、所有節點、所有條件、所有例外、所有命名，一次全端上來。結果讀者不是感到敬畏，而是直接腦袋關機。

這份 PDF 不該走那條路。

這份 PDF 的任務，是先讓大家看懂整體地圖，知道七大家族不是亂切，知道下面已經有子家族和子樹，知道這些細節經過壓力與邊界去磨，知道它不是只有母

表沒有內臟。但它不需要一次把整棵樹全部倒出來。

因為一張真正好用的地圖，不是把每一粒沙都畫上去，而是先讓你知道你在哪，下一步往哪走，哪些岔路最容易走錯。

所以這裡的策略應該是：

正文只放代表性子家族。
讓讀者看懂有 deeper structure。
而完整更操作化的層，留在 Router TXT 和後續更細的技術頁面裡。

這個節制，不是退縮。
這反而像一個冷靜的搞笑科學家終於學會了一件很難的事：

不是每一個實驗室抽屜都要在第一次見面時打開。

4.7 所以，第四章真正想講的是什麼

如果把這一章壓成一句話，我會這樣說：

七大家族是 Atlas 的骨架，子家族是 Atlas 開始長出肌肉的地方。

骨架讓你知道世界大概怎麼分。
子家族讓你開始知道，為什麼同樣在這一區，還會有不同的第一修法。

而最重要的是，這些子家族不是憑感覺亂長的。
它們後面有解析度紀律(Resolution Discipline)、邊界測試(Boundary Test)、壓力測試（Stress Test）、不准偷懶審核（No Laziness Audit）、以及「證據不夠就不要假裝很細」這種非常不討喜但非常重要的規則在盯。

這也是為什麼我會說，Atlas 不是一張只會命名的大圖。
它更像一個正在長大的 troubleshooting atlas。
上面那七個大標題是入口。
下面那些比較細的子樹，是它開始真的能走進真實案例、承受更細辨別、也承受更多打擊的地方。

而且，這裡其實還有一個更好玩的地方。
因為當你開始把子家族真的長出來，下一個讀者很自然會問的問題就會變成：

好，這樣看起來不只是在分類。
那它現在到底能做到哪裡，又還沒做到哪裡。

這也就是下一章要講的東西。

本章小結

這一章在做的事，其實就是幫子家族平反。

不是為了多切幾刀。
不是為了顯得更學術。
不是為了讓圖更複雜。

而是因為，真實世界很多誤修，發生在你已經站對樓層，卻還是在錯的房間門口。
所以七大家族下面，需要更細的子家族（Subfamily）、子樹（Subtree）、節點（Node），來幫你把第一刀下得更像樣。
同時，又要靠壓力測試（Stress Test）、邊界測試（Boundary Test）和解析度紀律（Resolution Discipline），避免這些細節長成裝飾用的假精準。

第五章

Atlas 現在到底做到哪裡，沒做到哪裡

5.1 Atlas 已經不是一張分類表

先講最重要的結論。

Atlas 現在已經不是一張單純的分類表。

如果它只是分類表，那它最多做到一件事，幫你把 bug 命名得比較漂亮。
這種東西不是沒價值，**但它很容易停在「喔，有趣」這個層次，離「我真的會拿來用」還差很遠。**

Atlas 目前公開的主頁，對自己的定位講得很清楚。它不是 checklist(清單)，也不是單純的 failure taxonomy（錯誤分類）。它把自己定義成一套 **debugging decision system（故障排除決策體系）**，目標是幫人和 AI systems 更穩定地做到幾件事：辨認 failure family，定位 failure region，找出 broken invariant，區分相鄰但容易誤判的區域，並決定什麼該先修、什麼不該先修。它甚至把整件事濃縮成一個很硬的核心句子：**Wrong first diagnosis creates wrong first repair.** **「錯誤的初步診斷，會導致錯誤的初步維修。」** 這代表 Atlas 的核心工作，不是「幫你起名字」，而是「幫你不要第一刀就切錯」。

Router 的凍結說明（Freeze Note）也把這件事說得很明白。它把 Router v1 定義成第一個正式的 **路由優先 TXT 包（route-first TXT pack）**，功能不是亂講一堆大原則，而是很具體地幫 AI systems 做幾件事：
分類最可能的主要家族（primary family）
指出最強的相鄰壓力（secondary / neighboring pressure）
說明為什麼主要切法成立
指出最可能壞掉的規則（broken invariant）
給出第一修法方向（first repair direction）
提醒最可能的誤修（misrepair）
並且在證據薄的時候保持誠實。

所以，如果用一句最不囉嗦的話來講：

Atlas 現在做的，已經不是「看到錯誤就幫你貼標籤」。
它已經在做「先把故障區切對，再把第一修法方向對準」。

這個差別非常大。
前者比較像會分類的學生。
後者比較像真的開始懂得怎麼看病歷的住院醫師。
雖然還不是主治，但你已經不會只把所有人都送去感冒門診。

5.2 它現在最成熟的地方，是路由優先(Route-first)的判讀層

如果現在要我幫 Atlas 找一個最成熟、最清楚、最能立刻拿去用的核心，我會選：

路由優先（Route-first）的判讀層。

因為主頁、Freeze Note、Atlas Hub、以及 TXT 本身，現在都在講同一件事，而且講得非常一致：

這套東西的第一強項，不是把所有東西一次修完，
而是幫你在最容易切錯的時候，先切得比較像樣。

Atlas Hub 現在給新手的入口也很誠實。它不是叫你先把所有理論看完，而是直接把「Try the compact router immediately」「See the fastest practical usage path」「Understand the product first」這幾個入口擺在最前面，然後才往後帶你去看 Atlas Final Freeze、Fixes Hub、Casebook、Adapter 等更深的層。這種導覽方式本身就在說一件事：

現在最成熟的，不是所有東西都已經完全展平。
而是這套 Router-first 的入口，已經成熟到足以先讓人實際開始用。

而這也是我覺得 Atlas 最聰明的地方。
它沒有一開始就裝成全能。
它先把最常害人白工的那一刀做好。
這很像一個搞笑科學家終於克服了自己想一次解完整個宇宙的衝動，決定先把最容易爆炸的那個閥門關起來。

5.3 它還不是自動萬能修復器

這一節一定要講，而且要講得很清楚。

Atlas 現在不是自動萬能修復器。

如果你把它想成一個「丟 bug 進去，然後它從根因到 patch 全部幫你收乾淨」的系統，那你對它的期待就已經超出目前公開邊界了。
這不是在幫它找藉口，而是 Freeze Note 本身就講得很清楚。它明說這份 Router v1 凍結文件的目的，是定義 Router 現在是什麼、它穩定了什麼、它被允許主張什麼。它也直接寫明：這個 Router 不取代 Atlas core，不取代 full adapter，不取代 casebook，也不取代更深的 WFGY repair 或 bridge work。換句話說，它不是整套宇宙，只是目前最正式、最可直接使用的第一個 TXT 路由產品。

這件事很重要，因為很多人一看到「能分類、能判 broken invariant、能給 first fix direction」，就會自動腦補成「那它應該已經會自動修」。
不，沒那麼快。
它現在比較像一個非常會看方向、很會阻止你修錯地方的技師。
它會指著車子說：

這台先看油路。
不是先拆引擎。
那個儀表板也別全信。
先把觀測打開。
先不要在車門上貼碳纖維貼紙。

但它不是現在就替你把整套引擎翻新完，然後順便幫你去監理站驗車。
這種誠實非常重要。
因為如果連這件事都不講清楚，外面的人第一次用完，就會覺得你在賣的是瑞士刀，拿到手卻發現比較像高級手術導航。
問題不是導航不好，問題是你一開始講得像是會自己開刀。

所以，這一章一定要讓讀者知道：

Atlas 很強的地方，是先把你帶到對的手術室。
不是現在就假裝自己已經是全自動機械手臂。

5.4 但它也不是只有分類，因為後面已經有修復導向結構（Fix-facing Structure）

這一節，是整章最容易被忽略、但其實最值得寫的一段。

因為 Atlas 現在雖然不是 full auto-fix，
但它也絕對不只是「分類完就散會」。

Fixes Hub（修復中心）的 README 寫得非常清楚，它把自己定義成 **Atlas 系統的修復導向延伸層（repair-facing extension layer）**。
原文幾乎是把角色分工講成四句非常漂亮的話：

Atlas 告訴你 failure lives 在哪裡。
Casebook 教你怎麼認那一道切口。

Adapter 幫 AI systems 在紀律下 route case。
Fixes layer 開始回答下一個問題：
正確路由之後，第一步應該先試什麼。

這段話的含金量很高。
因為它等於正式宣告：

Atlas 不是只負責命名。
它後面已經有一條明確的「從 diagnosis 走向 practical troubleshooting」**(從診斷走向實際修復)**的橋。Fixes Hub 甚至還直接說，這層東西同時是 atlas diagnosis、practical troubleshooting、deeper WFGY exploration、community growth **(地圖式診斷、實戰故障排除、深層 WFGY 探索、社群成長)**之間的重要橋樑。

所以你如果要用一句很精準、又不吹過頭的話來形容 Atlas 現況，我會這樣講：

它現在最完整的是 route-first diagnosis (路徑優先的診斷)
它還不是 full auto-fix。但它也已經不只是分類，因為 repair-facing 的結構已經長出來，而且已經有官方入口。

這種狀態其實很漂亮。
因為它代表整個系統不是卡在概念階段。
它已經開始長出「從地圖走向工具」的那段過渡層。
這段過渡層，就是最容易讓人開始認真看待一個專案的地方。

5.5 Demo 的存在，本身就是一種回答

如果有人還是不放心，心裡默默想：

「你講得很好聽，但這到底有沒有真的改變修法。」

那官方 Demo Pack 其實就是最直接的回答。

Flagship Runnable Demo Pack 的 README 一開頭就把目的講得很狠。它說，這些 demo 不是為了證明每一種 case 都已經全涵蓋，它們更重要的任務是證明一件事：

Atlas 不只是命名 failures。
它會改變第一修法。

這句話其實把第五章要講的東西整個釘住了。

如果 Atlas 只是分類學，那大家最多會覺得它有趣。
如果 Atlas 能讓不同路由決策，導向不同修復決策，大家才會開始感覺到它真的有用。官方 demo pack 甚至直接說，這整包東西存在的理由，就是要把這個 claim 變得可見，而且要用最小 runnable surface 的方式變得可見。

所以，Demo 的意義不只是「這裡有四個例子」。
Demo 的意義是：

它把 Atlas 從一張概念地圖，往「會影響你動手方式的系統」推進了一大步。

也就是說，現在 Atlas 已經站在一個很有意思的位置：

前面有穩定的 route-first 判讀。
中間有修復導向延伸層（Fixes layer）。
旁邊有 demo 去證明這不是空氣。
後面還有更深的 WFGY bridge 可以接。

這個結構，其實已經很不像一個「只是分類圖」的專案了。

5.6 它現在還沒做到哪裡，這反而是它值得信任的地方

很多人會覺得，講「還沒做到哪裡」會削弱作品。
我不這麼看。我反而覺得，對 Atlas 這種系統來說，敢把邊界講清楚，本身就是可信度的一部分。

因為 Atlas 現在的強項很清楚。
它已經有：

七大家族母表
穩定的 route-first contract（路徑優先的合約層）
broken invariant 的判讀邏輯
相鄰高壓邊界的核心切口
repair-facing 的橋接層

官方 MVP demo
以及 compact TXT product layer。

但它也還有一些地方，現在不該被吹成「已經全自動完工」。

例如：

它還不是完整 autonomous repair runtime。
它還不是任何 case 都能 node-level 一刀到位。
它下面的子樹（Subtree）雖然已經存在，但有些區域還處於持續加厚、持續被真實案例打磨的狀態。
它的 demo 現在是 MVP surface，不是 giant benchmark universe。
它的開源方式，也很明顯不是在說「我已經完成文明級終局」，而是「我把一套可用骨架公開出來，請大家拿真實世界來壓它」。

這種口氣，其實很難得。
因為它讓專案有一種很健康的氣質：

我不是在裝完成。
我是把已經站得住的核心先凍結，然後讓後面的成長進入有紀律的擴張。

Atlas Hub 也直接說了，專案已經不在「trying to find the core」的階段了，現在是在 structured extension、patching、teaching、productization、AI-facing reuse、router deployment、repair-surface thickening、bridge expansion、provenance strengthening（結構化擴展、補丁修復、教學傳承、產品化、面向 AI 的復用、路由器部署、修復面增厚、橋接擴張、溯源強化。）這個階段。這句話超級重要，因為它等於在說：

核心已經找到。
現在的問題不是有沒有核心。
是怎麼把核心長厚。

這種狀態，反而最值得人投入。
因為你不是在看一團還沒成形的霧。
你是在看一棟地基已經打好、骨架已經站起來、房間也開始有功能，但還歡迎大家一起裝修和打磨的建築。

5.7 所以，Atlas 現在最準確的定位是什麼

如果這一章最後要幫 Atlas 找一句最準的定位，我會寫成這樣：

Atlas 現在是一套已經跨過「單純分類表」階段的 route-first troubleshooting system（路徑優先的故障排除系統）。它最成熟的部分，是 failure routing、broken invariant 判讀、以及 first repair direction 的紀律化輸出。它還不是 full autonomous repair engine（全自動修復引擎）。但它也已經不只是分類，因為 repair-facing extension layer、official demos、以及更深的 WFGY bridge 已經明確長出來。

這句話，我覺得很準。
而且很有力量。

因為它沒有偷吹。
也沒有把自己講小。
它只是把目前真的站得住的地方，講到剛剛好。

這種「剛剛好」，比很多一開口就說自己要顛覆宇宙的技術作品，更能讓人放心。

而且說真的，一個搞笑科學家如果哪天突然不再亂丟閃電，開始認真標示「高壓危險，請勿徒手觸碰」，你反而會更想相信他的實驗室。

本章小結

這一章想講的事情，可以壓成三句話。

第一句：

Atlas 已經不是一張分類表。
它現在最成熟的，是 route-first 的判讀與第一修法方向。

第二句：

Atlas 還不是自動萬能修復器。
它現在不是那種丟 bug 進去、全部自己修好的系統。

第三句：

但它也不只是分類。
因為 repair-facing 的結構、官方 demo、以及更深的 bridge，都已經長出來了。

所以最準的理解方式不是「它是分類圖」。
而是：

它是一個已經開始把 diagnosis 變成實際 troubleshooting surface 的系統。

第六章

四個 demo，真正讓我確定這不是概念而已

6.1 一張地圖真正有價值，不是分得漂亮，而是會改變你下一步怎麼修

先講一個很簡單，但很多人會忘記的事。

一張 troubleshooting 地圖，如果只能幫你把錯誤命名得更整齊，它的上限其實不高。它可能很適合做簡報。也很適合拿來在社群裡讓大家點頭說「分類很細」。但真正的 debug 現場不吃這一套。現場只問一件事：

你到底有沒有讓我少走一段冤枉路。

官方 demo pack 的設計邏輯就非常聚焦。它不是 giant benchmark zoo，不是那種把一百種 case 全部排成動物園，讓你走到最後懷疑人生的東西。它明講，這是一個 compact, high-signal proof layer。目標不是規模，是清晰；不是展示宇宙萬象，而是讓一個最重要的 claim 變得可見，也就是 route-first repair 真的會改變第一修法，而且這件事已經能用最小 runnable surface 被看見。

這個設計其實很聰明。
因為它沒有先追求「全部都有」。
它先追求「最重要的那句話要先能被人看見」。

而那句話就是：

不同的故障，不該用同一種方式修。

聽起來像常識，真正做起來卻很少人做到。
因為表面上很像的 failures，常常很容易被粗暴地塞進同一個桶子裡。
一個答案很流暢但證據接錯的系統、一個 workflow 骨架沒有閉合的系統、一個符號容器先歪掉的系統、一個你根本看不見故障路徑的系統，表面都可以被罵成「系統很差」。Atlas demo 要證明的，就是這種說法太粗。你一旦把 case route 到正確的家族，第一個 repair move 就會改變。

所以這四個 demo 的價值，不是它們有四個。
而是它們像四把很短、但很準的手術刀。
每一把都在切同一件事：

你現在以為自己在修同一種問題，實際上不是。

6.2 這四個 demo，挑的不是「最帥的例子」，而是「最能看出差別的切口」

官方 demo pack 目前選了四個 demo，分別對應：

接地與證據完整性（Grounding & Evidence Integrity）
觀測與可診斷性完整性（Observability & Diagnosability Integrity）
執行與契約完整性（Execution & Contract Integrity）
表徵與定位完整性（Representation & Localization Integrity）

這個選法很有意思。
因為它不是單純挑「最熱門的四類」，也不是平均分配。
它挑的是四種只要你一看，就能明顯感覺到「原來第一修法真的會變」的切口。

官方 README 也直接把四個核心 proof 寫了出來：

Demo 1 證明重新接地（re-grounding）會改變第一修法。
Demo 2 證明 visibility uplift 要先於 answer repair。
Demo 3 證明 execution closure repair 要先於 reasoning-level repair。
Demo 4 證明 container repair 會改變後面能穩定什麼。

你如果把這四個 proof 放在一起看，就會發現它們像在一起喊同一句話：

不要一看到「怪怪的」就用同一把扳手。

這很像修車。
有些問題先補油。
有些先打開儀表板。
有些先補流程骨架。
有些先把變形的外殼或槽位校正。
如果你全部都拿一把「重寫回答」的大錘去敲，最後通常會得到一台更吵的車，而不是一台更好的車。

6.3 Demo 1：答案很順，但證據接錯了，所以第一步應該先重新接地，不是重寫語氣

Demo 1 的正式名稱是 **接地錨點恢復**（Grounding Anchor Recovery）。
它的主題一句話就講完了：一個答案看起來流暢，但它黏錯了證據錨點（evidence anchor）。官方 README 直接寫，這個 demo 想證明的是，failure 是 grounding-first，不是「模型笨」；第一個 repair move 應該是 re-grounding，而不是 style rewriting；而且一旦做了 evidence verification，repair path 就會改變。

這個 demo 很適合放在第一個，因為它特別像很多人真實遇到的情況。
AI 不是完全亂寫。它甚至寫得很順。
問題是，它把答案繫在錯的依據上。
這種時候，如果你誤判成「文風不好」或「推理不夠細」，你就會一直去補表面。
可是真正該做的，其實是把它重新接回正確的外部錨點。

官方 README 還特別說，Demo 1 是第一個 MVP 裡唯一有 live mode 的 demo，原因不是其他 demo 比較弱，而是 Demo 1 最適合直接讓讀者看到 baseline answer、repaired answer、anchor correction、以及 result movement。換句話說，這一題最適合用「眼睛一看就懂」的方式讓人感覺到：

原來有些錯誤，第一步真的不是改寫答案，而是先把答案接回現實。

如果要用人話把 Demo 1 壓成一句話，那就是：

不是所有錯答案都叫幻覺。
有些錯答案，真正先壞的是證據錨點。

這句話其實很有殺傷力。
因為它直接把一大堆平常被粗暴叫成 hallucination 的問題，重新分出層次。
也就是說，Demo 1 不是只在證明 F1 很重要。
它同時也在教育讀者：

連「錯答案」這件事本身，都不應該被同一種方式看待。

6.4 Demo 2：你不是先壞在答案，而是先壞在根本看不到自己壞在哪

Demo 2 的正式名稱是 **觀測優先**（Observability First）。
這個 demo 的 theme 非常漂亮，官方原文幾乎可以直接拿來當一句 punchline：

一個 failing workflow 之所以無法被正確修復，是因為它的 failure path 仍然是隱藏的。
它想證明的是，第一個 failure 是可診斷性（diagnosability），正確的第一修法是 observability insertion，太早修 answer 是錯的，visibility 會改變整個 repair landscape。它的主 lesson 更簡單：有時候第一修法不是 repair the system，而是 make the system visible。

這個 demo 非常像工程師的惡夢。
你知道系統有問題。
你甚至知道它應該有地方在漏。
但你就是看不到漏點。
這時候你如果開始急著「修系統」，通常會很容易修成一種表面忙碌症。
你補一點這個，調一點那個，最後得到很多 commit 和很多疲勞，卻沒有真正得到診斷能力。

Demo 2 把這件事切得很乾淨。
它在說：

有些系統不是先壞在功能本身。
它是先壞在你根本無法看清功能怎麼壞。

這句話其實很有哲學味，但也非常工程。
你不把觀測窗打開，
就像拿黑布蓋著儀表板修車。
你不是不能修，
但你多半會修得非常有創意，然後非常偏。

這也是為什麼這個 demo 很能打。
因為它把很多人心裡模糊知道、但沒說清楚的一件事具體化了：

不是所有第一修法都應該直接落在答案或 output。
有些第一修法，應該先落在「讓系統變得可見」。

這種想法一旦被看見，讀者就會開始真正理解，Atlas 不是單純在分家族，而是在改變「下一步要先做什麼」這件事。

6.5 Demo 3：它不是不會想，而是流程骨架根本還沒閉合

Demo 3 的正式名稱是 **執行閉合（Execution Closure）**。
這個 demo 的主題寫得非常直白：系統失敗，是因為 execution skeleton closure 在 reasoning quality 甚至還沒來得及重要之前，就先壞了。它想證明的是，問題不主要是 memory 或 reasoning，問題是在 readiness、ordering、bridge 或 liveness；正確的第一修法是 execution closure repair，而且 system structure 可以先於 model reasoning 成為限制因子。它的主 lesson 更短，也更狠：

有些失敗是 workflow skeleton 造成的，不是 intelligence quality 造成的。

這句話其實非常有殺傷力。
因為現在很多人一看到 AI 任務沒做完、agent 沒跑順、多步流程卡死，第一反應就會怪模型不夠聰明。
但 Demo 3 在說：

先等一下。
也許不是腦子先壞。
也許是骨架先壞。

這就很像一台車不是引擎沒力，
而是傳動鏈根本沒接起來。
你一直逼引擎輸出更高馬力，結果輪胎還是沒收到。
這不是愚蠢，這是浪費。

Demo 3 最好的地方，就是它把一個很容易被人格化的問題，重新拉回系統結構層。不是「AI 為什麼那麼笨」。
而是「這條 sequence 根本沒被正確閉合」。

這種視角一出來，很多 debugging 的焦點就會瞬間轉移。
你會開始看：

是不是 ordering 錯了。
是不是 readiness 還沒滿足。
是不是 bridge 沒接上。
是不是 liveness 根本掉了。

也就是說，Demo 3 在提醒你：

有些系統不是要先補推理，
而是要先補骨架。

這對很多做工作流、多步 agent、pipeline orchestration 的人來說，會非常有感。
因為你終於不是再被迫在「模型不夠強」和「prompt 不夠好」之間二選一了。
你開始有第三條路：

也許真正該修的，是執行閉合。

6.6 Demo 4：裡面不一定壞，是裝載思考的那個盒子先歪了

Demo 4 的正式名稱是 **容器保真**（Container Fidelity）。
這個 demo 的主題非常有特色，官方原文幾乎就是整個 Atlas 裡最有辨識度的一把刀：

一個任務看起來像 reasoning failure，但真正先壞的是 structure carrier。它要證明的是，問題不是 purely progression-first；symbolic 或 formal containers 可以在 reasoning 成為主要問題之前先壞掉；第一修法應該打在 descriptor fidelity 或 formal adequacy；而且 container-first repair 會改變系統之後能穩定做到什麼。它的 main lesson 更像一句會被讀者記住的話：

有時候系統不是因為它不會想而失敗。
有時候它失敗，是因為裝載思考的那個盒子已經先壞了。

這一句，真的很強。
因為它把「表徵與定位類（Representation / Localization）」這個很多人第一眼很難抓的家族，一口氣變得非常人類可懂。

你想一下。
如果一個人講話前，嘴巴裡先塞了三層紗布，
你不能直接說他邏輯不好。
也許他根本就只是講不出來。
容器、載體、孔位、格式、版面、符號結構，這些東西如果先歪掉，裡面的 thinking 不一定有錯，但它已經沒辦法被穩定承載出來了。

這個 demo 特別會打到幾種人，官方也列得很準：

結構化輸出的人
JSON 與 schema 使用者
程式與符號推理使用者
OCR 或版面敏感管線使用者
以及那些對 Atlas 最有辨識度那一刀有興趣的人。

也就是說，Demo 4 不只是展示 F7。
它同時還在教育讀者一件很重要的事：

不是所有看起來像「推不動」的問題，都該先怪 reasoning。
有些系統，是表達那個思考的容器先垮了。

這就是為什麼我會說，Demo 4 很像 Atlas 的招牌刀法。
因為它切到一個很多系統都中過，但平常很少被好好命名的地方。

6.7 為什麼這四個 demo 放在一起，會讓人開始真的相信 Atlas

如果只看單一 demo，你可能會覺得：

好，這是一個不錯的例子。
但那也許只是這一題剛好適合。

可是四個 demo 放在一起，味道就完全不一樣了。
因為它們不是在重複同一種 proof。
它們是在從四個方向，同時壓出同一個結論：

Demo 1 說，先重新接地，不要先修表面。
Demo 2 說，先把系統變得可見，不要急著補答案。
Demo 3 說，先補執行閉合，不要先怪模型不夠聰明。
Demo 4 說，先看容器是不是歪了，不要直接把所有東西都丟去 reasoning。

你會發現，這四個 demo 其實共同在打同一種壞習慣：

一看到問題，就急著用最順手的那把工具修。

而 Atlas 最值錢的地方，就是它逼你停一下。
停那一下，先 route。
先承認故障區不同。
先承認第一修法不該亂共用。
先把最像、但其實錯的相鄰故障區排掉。
然後再動手。

這就是為什麼官方 demo pack 不追求 giant benchmark zoo，而是追求最小但清楚的 runnable proof。
因為它不是想證明「我有很多範例」。
它是想證明：

只要你真的接受不同失敗需要不同第一修法，Atlas 就會開始變得非常實際。

而且 README 也特別把目前 demo pack 的型態說清楚了。現在第一批 flagship demos 已經在 usable MVP state；Demo 1 具備 live notebook 與 replay support，Demo 2、3、4 則以 replay-first 的 v2 notebook 作為推薦入口。
這不是偷懶，而是 deliberate MVP teaching decision，也就是刻意選擇最能

把教學模式講清楚的方式。官方還直接說，replay mode 是最重要的 public mode，使用者不需要 API key，就能檢視 case、baseline、atlas route、first repair move、before / after outputs、以及 changed explanation；一個人就算不跑 notebook，也應該能理解 demo 想證明什麼。(GitHub)

這個態度其實很加分。因為它再一次表明，Atlas 不是要靠炫跑環境來取勝。
它要的是：

你不用先配好一整套設備，也能看懂這套地圖到底有沒有改變修法。

這就很實在。

6.8 Demo 的真正作用，不只是證明現在，更是替後面開路

這裡還有一個很值得講的地方。

官方 README 明講，這四個 demo 不只是 proof，它們也是 template。
它們是往後 community growth 的種子資產，可以往 Colab、JSON fixtures、prompt packs、workflow reproductions、benchmark reruns 這些方向延伸。
它甚至把這一點列成 demo pack 想證明的第四件事：一旦第一批 flagship demos 存在，社群擴張就會容易很多。

這件事很重要。
因為它表示 demo 不是一場煙火。
不是點完一次、拍完照片、社群鼓掌三分鐘就沒了。
它更像一批「官方先例」。
你先把最小、最清楚、最能教人的版本立起來，後面大家才知道可以怎麼照這個模板去換模型、換輸入、換 fixture、換 prompt、延長 repair path、拿自己的 case 來對照。

換句話說，這四個 demo 的價值不只是「現在能讓人相信」。
它們同時還在幫 Atlas 準備一種更長尾的成長方式。
也就是說，demo 不是展示櫃。
它開始像一套標準件。

這個味道其實很關鍵。
因為當一個系統開始有「大家可以照著擴」的 demo surface，它就開始比較不像一個漂亮研究點子，而比較像一個真正準備進入社群迭代的產品核心。

6.9 所以，第六章真正想說的是什麼

如果把這一章壓成一句話，我會這樣寫：

這四個 demo 的真正價值，不是它們有四個。
而是它們一起讓人看見：Atlas 不是只會命名 failures，它真的會讓第一修法改變。

這一句，才是整章的中心。

Demo 1 讓你看到，先重新接地。
Demo 2 讓你看到，先把系統變得可見。
Demo 3 讓你看到，先補執行閉合。
Demo 4 讓你看到，先修容器，不要急著怪 thinking。

四個 demo，像四個不同方向的實驗探針。
它們共同證明的不是「Atlas 很厲害」。
而是更實在的一件事：

如果你接受不同故障區要有不同第一修法，那 Atlas 這張地圖就開始真的有用。

而這個 moment，很重要。
因為從這裡開始，讀者通常會自然冒出下一個念頭：

好，這樣我大概信了。
那我可不可以自己拿一個 case 來試。

這，也就是下一章要接的地方。

本章小結

這一章做的事情，是把 Atlas 從「聽起來有道理」拉到「看起來真的能用」。

四個 demo 不在證明所有東西都做完了。
它們在證明一件更重要的事：

Atlas 不只是在替 failures 命名。
它真的會改變你下一步怎麼修。

這就是為什麼 demo pack 值得存在。
不是因為它有四個 notebook。
而是因為它把那句最關鍵的話，變成了可以看見的東西。

第七章

一份 TXT，怎麼讓你 60 秒就開始試

7.1 你不用先懂完整 Atlas，才能開始用

這件事一定要先講。
因為很多人一看到 repo、看到七大家族、看到子家族（Subfamily）、看到子樹（Subtree）、看到 broken invariant、看到 relationship matrix，腦袋就會自動切成兩種模式：

第一種是「哇，好完整」。
第二種是「哇，我完了」。

第二種很常見。
因為大家會直覺以為，要開始用這種東西，自己必須先把整份 Atlas 全讀完、把七大家族背起來、把每個邊界都搞懂、最好還要先修完一門「故障分類導論」。
其實不是這樣。

官方 Usage Guide 的寫法非常務實。它一開始就不是叫你去啃所有理論，而是直接說，這個 Router 是一個實用入口，你可以先把 pack 丟進模型，再拿真實 case 去跑，然後先看 output contract 裡最重要的幾個欄位。換句話說，它不是先假設你已經是 Atlas 神官，才允許你試用。它反而是在設計上就承認：**很多人是從「我手上有一個亂七八糟的 case」開始接觸它的。**

這個順序很重要。

因為如果你把 Atlas 當成一門要先修完才能用的學科，它很快就會被很多人放進「以後有空再研究」的抽屜。但如果你把它當成一張路由地圖，你就會接受一件比較健康的事：

我現在不需要懂完整城市規劃，我只需要先知道，我是不是走錯方向。

這個心態一對，使用門檻就會瞬間下降很多。

所以如果你現在手上已經有一個亂掉的 bug、一段你懷疑 AI 一直修表面的對話、一個看起來到處都像問題的 workflow、一個你越看越想把筆電闔上的怪 case，其實你已經可以開始試。不用等到你對七大家族如數家珍。
不用等到你把所有子家族全部背熟。也不用等你內心長出一位 full-time 故障分類學家。

你現在只需要做對一件事：

先把這份 TXT 當成路由起點，而不是當成最終審判。

7.2 最短使用方式，其實比你想像中更短

這裡直接講最實用的版本。

如果你今天就想試，最短流程其實可以很短。
官方 Usage Guide 給的骨架，也是這樣設計的。

第一步，拿到 Router TXT

先把 Router TXT 檔準備好。
這個檔案本身就是一個為 route-first diagnosis（路徑優先診斷）設計的 compact pack（壓縮包）。它不是 full repair engine（全自動修復引擎），也不是 autonomous agent(自主智能體)。它更像一個嚴格的故障分流框架，會要求模型先做 family-level 判讀，再根據證據決定是否往更細的 best current fit 前進。

第二步，把 TXT 貼進你正在用的模型

官方說明列出的典型工具包括 ChatGPT、Claude、Gemini、Copilot Chat、Cursor，或其他可以吃長文字上下文的模型介面。重點不是品牌，而是你要有一個足夠強、足夠穩的大型語言模型（LLM）來讀這份 pack。

第三步，貼一個真實 case

這裡最重要的是「真實」。
不要先拿你已經整理成教科書格式的 case。
如果你真的要測它，就拿你手上那個亂糟糟的東西來。
例如：

一段出錯紀錄
一組 logs
一段你和 AI 來回三輪還越修越怪的對話
一個 workflow 的錯誤現象
一段 prompt 加 output 加你的觀察

Atlas 本來就是設計來面對這種「症狀很多、表面很亂、相鄰 failure 很像」的情境。主頁也明講，它不是給玩具故障用的，而是給 complex AI failures 用的 decision surface。

第四步，用最小 prompt 啟動

Usage Guide 其實有提供很精簡的啟動方式，精神上就是：

請用 Router 的 output contract 來 route 這個 case。
先給 primary family、secondary family、why primary not secondary、broken invariant、best current fit、first fix direction、misrepair risk、confidence、evidence sufficiency（主族系、次族系、為何選主不選次、被破壞的不變量、目前最佳擬合、首要修復方向、誤修風險、信心度、證據充分性。）

這個步驟很重要，因為它在一開始就限制模型不要亂飛。
不是叫它寫小作文，
不是叫它直接 patch，
不是叫它用一段看似深刻的長文把你帶進另一個洞。
而是先照 contract 做 route。

第五步，先看關鍵欄位，不要貪心

這一點後面會再講一次，但先在這裡提醒。
你第一次使用，不需要每個欄位都拿放大鏡研究。
先看幾個最重要的欄位，就已經足夠幫你感受 Atlas 的味道了。

所以，整個流程其實可以被壓得很短：

拿 TXT
貼進模型
貼你的真實 case
要求照 contract route
先讀幾個核心欄位

這真的是可以 60 秒內開始的。
不是 60 秒內變成 debugging 之神，
而是 60 秒內開始從亂打一通，變成比較有方向地看問題。

7.3 第一次使用時，你只要先看這幾個欄位就夠

這一節非常重要。
因為很多人第一次接觸這種結構化輸出，最容易犯兩種錯：

第一種是完全不看欄位，只看模型最後那段總結。
第二種是每個欄位都想一次看懂，結果直接 overload。

我建議最健康的做法是：

第一次，先只看六個欄位。

1. 主要家族（Primary Family）

這是最重要的第一眼。
它在回答的不是「最後結果像什麼」，而是「目前最像哪一種先壞方式」。
這一欄會直接決定你接下來要不要先往資料、推理、狀態、執行、觀測、邊界、容器這些不同方向看。Usage Guide 本身也把它放在第一位來讀。

2. 次要家族（Secondary Family）

這欄不是裝飾。
它是在提醒你「最像、但還沒贏的那個相鄰故障區」是誰。
也就是說，它在幫你標示邊界壓力。
這很有用，因為很多 case 真正危險的地方不是一眼就完全看錯，而是卡在兩類之間。Atlas 的精神本來就很重視 neighboring pressure。

3. 為什麼是主要家族，不是次要家族（Why Primary Not Secondary）

這欄是整份輸出裡最有靈魂的地方之一。
因為它逼模型不能只丟一個標籤，而要交代「為什麼不是那個看起來也很像的相鄰家族」。這一欄如果寫得有內容，你通常就會知道模型到底是在亂貼標籤，還是真的有做 boundary cut。Usage Guide 也明講，讀輸出時，這一欄應該很前面就看。

4. 壞掉的規則（Broken Invariant）

這一欄很像故障的核心句。
它不是在描述表面症狀，而是在說「到底哪條本來應該穩定成立的規則，現在不成立了」。這欄一旦對，你看問題的角度會整個轉掉。
例如你不再只說「它怪怪的」，而會開始說「它現在是 evidence coupling broken(證據耦合斷裂)」或「execution closure broken(執行閉環斷裂)」或「continuity broken(連續性斷裂)」。這就是從症狀語言，開始進入故障語言。
Atlas 主頁本來就把 broken invariant 當成核心輸出之一。

5. 第一修法方向（First Fix Direction）

這欄是最實用的。
它在回答：

那我現在第一步先做什麼。
不是所有事情都做。
不是把整個系統重寫。
不是開啟 debug 宗教儀式。
而是先做哪個方向的修補比較合理。

這欄如果跟你的直覺不一樣，先不要急著不爽。
很可能那個不一樣，正是你現在需要它的地方。

6. 誤修風險（Misrepair Risk）

這欄非常 underrated。
它不是在唱衰你。
它是在提醒你：如果現在切錯、修錯，最可能會怎麼歪。
有時候這比 first fix direction 還值錢，因為它會讓你少掉一段原本很可能會白花的時間。

如果你第一次只看這六欄，通常已經足夠讓你感受到 Atlas 跟一般「請 AI 幫我 debug」的差別。
因為它逼你先站到一個比較像**分流與定位**的角度，而不是一開始就跳進 patch。
這個差別，真的會讓心情比較穩。

7.4 什麼情況最適合先拿 TXT 來試

這一節，我想寫得很實用。
因為不是每一種 bug 都需要把 Atlas 搬出來。
有些 bug 很簡單，你一看 log 就知道少一個逗號，那就真的不用叫出整套故障地圖。

但有幾種情況，特別適合你先拿 TXT 試。

第一種，症狀很多，而且彼此很像

這種 case 最適合。
你看到的是一團霧，不是一條清楚錯誤。
每一段看起來都像問題。
你已經分不清楚哪個是根因，哪個是連帶。
這時候 Atlas 很適合出場，因為它本來就不是為單點 typo 而設計，而是為 complex failures 設計。

第二種，你懷疑 AI 一直在修表面

這種也非常適合。
你已經感覺到某種怪事：AI 不是沒做事，它很忙，但你心裡一直有一種「你現在修的可能不是最早先壞的地方」的感覺。
恭喜，這就是 Atlas 最該出場的瞬間。

第三種，你自己也開始不確定要先看哪一層

有時候不只是 AI 迷路，你自己也迷路了。
你在猶豫這到底比較像資料問題、格式問題、流程問題、狀態問題、邊界問題，還是根本是看不見問題。
這時候比起再加一段 prompt，先做 route 通常更有幫助。

第四種，多輪對話之後，問題越修越多

這也是非常典型的 Atlas 適用情境。
你不是完全卡在第一步。你是已經修了三輪、五輪、八輪，然後開始懷疑自己是不是在幫 AI 一起蓋一條錯路。
這時候最需要的，通常不是再第九輪 prompt engineering，而是回頭先問：我們是不是第一刀就切錯。

第五種，你要把 case 拿去跟別人討論

這點也很實用。
因為 Atlas 給你的不是只有「我感覺是這樣」，而是一套相對結構化的 route 語言。你可以把 primary family、secondary family、broken invariant、first fix direction 帶去跟別人討論，這會比單純說「我覺得 AI 很怪」強很多。

換句話說，TXT 最適合的不是「所有 bug」。
它最適合的是那些已經有點混、已經有點亂、已經有點懷疑是不是修錯層的 case。
也就是說，它最擅長的不是代替常識，而是幫你把**快失去方向感的 debugging** 拉回可路由狀態。

7.5 什麼情況不該把它當成萬能解答

這一節也要講，因為健康的使用方式，不只是知道什麼時候該用，也要知道什麼時候不要亂用。

第一，不要把它當成全自動神諭

這不是神諭。
它不是那種你丟一段東西進去，它就替你宣布宇宙真相。
它是 route-first surface。
它的長處是讓你比較不容易第一刀切錯，不是保證每一題都一步封神。Freeze Note 與 TXT 本身對這點都非常節制。

第二，證據很薄的時候，不要硬裝很細

如果你手上只有一行模糊描述，沒有 log，沒有上下文，沒有前後關聯，卻硬要它切到很細的節點（Node），那不是 Atlas 的問題，那比較像你在逼導航替你預測宇宙。Router TXT 自己就有解析度紀律（Resolution Discipline），證據薄時能停在家族層就停，這反而是成熟。

第三，不要把它當成逃避思考的工具

這個也很重要。
有些人一看到有 structure，就會很想把自己的腦袋外包。
不是不行，但這樣你很快就會把任何輸出都當真。
更健康的做法是：把 Atlas 當成你的故障路由顧問，而不是你腦中的執政黨。

第四，不要把 cosmetic cleanup 當成 first repair

TXT 裡面對 misrepair 的提醒很嚴格，其中一種就是把 first fix 漂成 cosmetic cleanup，也就是先去修那些看起來漂亮、其實沒碰到根因的東西。這種情況在現實世界超常見，所以你一旦看到 first fix direction 指向比較不直覺的地方，不妨先停一下，不要立刻逃回最順手的表面清理。

簡單講，Atlas 最不適合被用成一種「我不想想了，你替我判」的工具。
它更適合被用成一種「幫我先把方向拉正」的工具。
這兩者看起來很像，實際上差很多。

7.6 如果你今天就要試，我會建議你這樣開始

如果我今天真的坐在你旁邊，而不是在這裡寫字，我會這樣叫你做：

先不要挑最乾淨的例子。
也不要挑那種你已經知道答案、只是想測它會不會答對的例子。
那種測法雖然也有意思，但比較像考試，不像真實 debug。

我會叫你挑一個你最近真的有點煩、真的有點亂、你也懷疑自己是不是一直在修表面的 case。
把 TXT 丟進去。
把 case 貼上去。

叫它先 route。
然後你先只看六個欄位：

主要家族（Primary Family）
次要家族（Secondary Family）
為什麼主家族不是次家族（Why Primary Not Secondary）
壞掉的規則（Broken Invariant）
第一修法方向（First Fix Direction）
誤修風險（Misrepair Risk）

不要先叫它寫 patch。
不要先叫它優化。
不要先叫它幫你「完整解決」。
先 route。
只要你做過一次這種 route-first 進場，你大概就會明白 Atlas 跟一般 AI debug prompt 最大的差別在哪。

差別就在於，它不是一開始就衝去幫你動手。
它先逼你承認，你可能連樓層都還沒站對。

而這個停頓，非常值錢。

7.7 所以，第七章真正想講的是什麼

如果把這一章壓成一句話，我會這樣講：

Router TXT 最有價值的地方，不是它很長，而是它讓你不用先讀完整座 Atlas，也能開始用正確的方法進場。

這是非常重要的。

因為很多好的系統，最後都死在「入口太重」。
明明核心很好，
但第一次接觸的人要先翻過三座山、四個術語表、五份結構圖，才摸得到門把。
Atlas 現在公開的 TXT 與 Usage Guide，做的其實是相反的事。
它是在努力把那個「第一次進場」壓到足夠短，足夠實用，足夠誠實。(github.com)

你不用先變專家。
你也不用先對所有子家族都熟。
你只需要願意先承認一件事：

現在最該做的，可能不是馬上修。
而是先看自己是不是修錯方向。

只要你願意這樣開始，TXT 就有價值了。

本章小結

這一章在做的事很實在。

不是教你怎麼瞬間成神。
而是教你怎麼**真的開始用**。

重點不是把整個 Atlas 一次吃下去。
而是拿 TXT、貼進模型、貼一個真實 case，然後先看幾個最重要的欄位，讓自己從「亂打一通」進入「先 route 再動手」的狀態。

這個轉變，聽起來不華麗。
但在真正的 debug 現場，常常比多一個 patch 更有價值。

第八章

FAQ：我不是工程師，我也能用嗎

8.1 我不是工程師，我也能用嗎

可以。
而且我會說，這其實是 Atlas 很值得的一個地方。

因為它現在最成熟、最公開、最能直接拿來用的那一層，不是什麼需要先裝完整環境、先配一堆服務、再跑半天 benchmark 的重型 machinery。
它最成熟的入口，反而是一個 Router TXT，搭配一個很短的使用路徑。主頁寫得很直接，這套東西是給 vibe coders、AI app builders、workflow builders、agent builders、還有在複雜 AI 系統裡工作的人用的，不是只給某一種純正工程血統的人。Usage Guide 也不是先假設你熟悉全部母框架，而是直接教你怎麼貼 TXT、怎麼貼 case、怎麼照 output contract 讀關鍵欄位。

換句話說，Atlas 的設計不是「先考過期中考，才有資格進場」。
它比較像一張你可以先拿來找方向的地圖。

你不需要先懂每個 broken invariant 的完整家譜。
你也不需要先把七大家族（Seven Families）全部倒背如流。
你甚至不需要先知道每個子家族（Subfamily）和子樹（Subtree）到底差在哪。
你只要有一個真實 case，願意先承認「我可能修錯方向」，那你就已經有資格開始用了。

這裡有一個很重要的心理轉換。
很多人會把這種系統想成「專家工具」，於是覺得自己還沒準備好。
但 Atlas 現在最實用的那一層，其實更像是**分流工具**。
分流工具最重要的，不是你有多專家。
而是你要先別跑錯科。

所以如果你不是工程師，但你有在用 AI 寫流程、做自動化、做資料處理、做 agent、寫 prompt、跑 RAG、修 workflow，然後已經體驗過那種「AI 很努力、很有自信、但越修越像在亂跑」的感覺，那 Atlas 其實就是寫給你看的。

8.2 它是不是會自動幫我修好

不會。
至少，現在不要把它想成這個。

這句一定要講清楚，因為很多工具最容易翻車的地方，就是大家對它的預設想像太超前。Atlas 目前最強、最穩、最可直接使用的能力，是 route-first diagnosis，也就是先幫你看現在比較像哪一個故障區、哪個相鄰區看起來很像但可能是錯的、哪條規則（broken invariant）比較像先壞了、以及第一步修法方向該往哪裡走。主頁、Freeze Note、Usage Guide 都反覆在做這件事，而且

Freeze Note 很明白地把邊界講出來，Router v1 是正式的 route-first TXT pack，但它**不是** full Atlas、不是 full repair engine、也不是 autonomous diagnosis closure。

所以如果你問：

它會不會自動幫我從頭到尾修好。

最誠實的答案是：

現在不要這樣想。

比較準確的理解方式是：

它會幫你少走一大段本來很可能會白花的路。
它會幫你先別在錯的樓層狂打補丁。
它會幫你把「先修哪裡」這件事變得比較有紀律。

說白一點，它比較像一個很會看方向、很會叫你先不要亂拆的技師。
它不是現在就替你把整台車全修完的自動工廠機械手臂。

但這裡也有一個重要的補充。
雖然它不是 full auto-fix，**可它也已經不是「分類完就散會」**。Fixes Hub 很明確把自己定義成 Atlas 的 repair-facing extension layer，也就是 route 對之後，後面其實已經有一層是專門在回答**「那第一步先試什麼 repair」**的。
這就表示 Atlas 不是只有命名，而是已經開始往實際 troubleshooting surface 走。

所以，如果你想要一句最準的答案，我會這樣說：

它現在不會自動全修好。
但它會讓你更有機會先修到對的地方。

這個差別，看起來不花俏，但在真實 debug 現場，其實非常值錢。

8.3 我不懂七大家族全部細節，也能開始嗎

可以，而且老實說，**大多數人本來就不需要一開始全懂。**

很多人接觸一套系統時，最容易有一種「我還沒讀夠，所以不該用」的心理。
這種想法很認真，也很可愛，但在 Atlas 這種工具上，反而可能讓你錯過它最實用的地方。因為 Atlas 現在設計得最成熟的那一層，就是讓你可以**先用，再慢慢長理解**。

主頁的「Start here in 60 seconds」和 Usage Guide 的整體寫法都在傳達同一件事：

你不用先深懂整個系統，才能開始。
你只需要先知道最短入口，然後讓模型照 output contract 給你一個 route-first 的判讀，再按照建議順序讀幾個關鍵欄位。Usage Guide 還特別提醒，第一次讀輸出時，先看 primary_family、再看 why_primary_not_secondary、再看 broken_invariant、最後看 first_fix_direction。這就是在承認，使用者第一次不需要把整包都吞下去。

你可以把七大家族想成大地圖。
第一次上手，你不用先把所有街道巷弄都背起來。
你先知道自己現在比較像在台北，不像在高雄，這就已經很有用了。
接著你再慢慢學哪一區容易塞車，哪一區最容易走錯交流道，哪一區其實長得很像別區。這就像 Atlas 裡從母家族走向子家族(Subfamily)、子樹(Subtree)、再到更細節點（Node）的路。
不是第一次就要全通關。

所以，如果你現在只記得一件事，那就記這件事：

先 route，先知道自己在哪一區。
比一開始就想背完整本 Atlas，更重要。

而且這樣反而會讓後面的理解更快。
因為你不是在空背結構，你是在拿自己的真實 case 去對照那個結構。
人腦對這種東西記得特別牢，畢竟痛過的 bug 都比較容易進入長期記憶。

8.4 如果兩個家族都很像，我該怎麼辦

這是個超級好問題。
而且我會說，能問出這題的人，通常已經開始抓到 Atlas 的精髓了。

因為 Atlas 最大的價值之一，本來就不是把所有 case 裝成百分之百一眼明確。它真正成熟的地方，是它很重視**相鄰又誘人的錯區**。主頁明講，這套系統不只是在問哪裡出錯，還在問哪個 neighboring region 很像、但其實是錯的。Freeze Note 與 Router 的 output contract 也都要求模型不只要給 primary family，還要給 secondary family，並且說明 **why primary not secondary**。這代表 Atlas 從設計上就承認：很多 case 本來就會卡在邊界。

所以，當你看到兩個家族都很像，第一個要做的不是慌。
第二個要做的也不是硬裝自己很懂。

更好的做法是這樣：

先看主要家族（Primary Family）和次要家族（Secondary Family）是誰。
再看「為什麼主要家族不是次要家族（Why Primary Not Secondary）」這一欄到底講得有沒有內容。再看 broken invariant 是否真的比較偏向其中一邊。
最後再看 evidence sufficiency，也就是目前證據是不是足夠支撐這個切法。

如果你看到輸出裡自己都透著一股「我其實也沒很確定，但我硬要演得很確定」的味道，那你就要小心了。

因為 Router TXT 自己最重視的，就是**證據薄時不要裝細**。
它有很明確的解析度紀律（Resolution Discipline），也就是說，在證據不足時，停在家族層（family-level）反而是成熟；只有在直接證據真的夠的時候，才往更細節點（node-level）走。它甚至把「假裝精準」「主題相似就硬跳 node-level」「把 no-fit 當逃生門」這些都列成 fail condition。

所以，真正成熟的答案通常不是：

「我百分之百知道，就是這個。」

而是：

「現在比較像 A，B 也有壓力，但依照 broken invariant 和現有證據，我先把第一修法落在 A 的方向；如果新 log 或新 trace 出來，切法可能會再變精細。」

這種口氣，才是 Atlas 喜歡的。
也就是說，**承認邊界壓力，不是失敗。**
邊界壓力本來就是這套地圖設計時最在乎的東西之一。

8.5 我只有一個很簡單的小 bug，也有必要用嗎

這題答案比較微妙。
不是單純的 yes 或 no。

如果你的 bug 真的是那種一看就知道少一個括號、少一個 import、環境變數拼錯、JSON 多一個逗號，那你當然不需要每次都把整張 Atlas 搬出來。
那樣就像明明只是鞋帶鬆了，卻先叫一台拖吊車來，畫面會有點浮誇。

但如果你的「小 bug」，其實已經開始出現以下任何一種味道：

AI 已經修了一輪，卻沒修對。
每補一次，症狀就換一個地方長。
你開始懷疑自己修的是表面，不是根因。
看起來像記憶問題，但又很像資料問題。
看起來像格式問題，但又很像流程問題。

那它就不一定真的是「小 bug」了。
它比較像披著小 bug 外套的迷路現場。

主頁本來就把 Atlas 的適用時機寫得很明白：當 AI debug 開始變得昂貴、模糊、又很自信；當 conversation 聽起來很聰明，但 repair direction 一直 drift；當你開始在 debug 症狀，而不是 failure region；當每個新 fix 都讓系統更亂，這些都是 route-first system 應該出場的時刻。

所以我的建議會很實際：

真的一眼就知道的小 bug，先用常識。
但只要你開始覺得「這問題怎麼修著修著味道不對」，Atlas 就值得出場。

因為它的價值不是在於 bug 大不大。
而是在於**你有沒有開始失去方向感**。

8.6 如果 demo 我看不懂，還有必要往下嗎

有，而且很正常。

先說一個讓人比較放鬆的事。
官方 demo pack 自己就不是設計成「只有把 notebook 全跑完的人才算懂」。
README 很直接地說，replay mode 是最重要的 public mode；使用者就算不執行 notebook，也應該能看懂每個 demo 想證明什麼，因為重點在 case、baseline、atlas route、first repair move、before / after outputs，以及 changed explanation。這其實是在設計上替很多人留了門。

所以，如果你看到 demo 頁時有一種：

「等等，這個 notebook 和 fixture 我現在不太想打開」

的感覺，完全沒關係。
你可以先這樣看：

先看 demo 在證明哪個 family 的第一修法會不同。
再看它的人話版主題。
例如：

Demo 1 是「答案很順，但證據接錯，所以先重新接地」
Demo 2 是「先把系統變得可見，不要急著補答案」
Demo 3 是「先補執行閉合，不要先怪它不會想」
Demo 4 是「先修容器，不要把所有怪事都丟給 reasoning」

這樣看，其實就已經很夠讓你抓到 Atlas 的核心主張了。
你不是在念實驗手冊。
你是在看四個切法不同的故障故事。

等你真的有興趣，或者你手上的 case 剛好很像其中一種，再回去看 notebook、看 replay、看 before/after。
這樣比較健康。

所以，demo 看不懂，不代表你不適合用 Atlas。
它只代表你現在先從更適合自己的入口進場。
主頁本來就設計了很多入口：Router TXT、Usage Guide、Official Demos、Fixes Hub、Atlas Hub。不是每個人都要從同一扇門進。(GitHub)

8.7 我可以拿自己的真實 case 來試嗎

可以，而且這其實是最推薦的做法。

因為 Atlas 最不適合的測法之一，就是只拿那種你已經知道標準答案、整理得乾乾淨淨、像教科書範例一樣的題目來「考它」。
那種測法也有意思，但比較像考試，不像真實 debug。

主頁本來就是寫給真實開發現場的。它講的不是 abstract elegance，而是那些 AI debug 開始變得 expensive、vague、self-confident、repair direction keeps drifting、every new fix makes the system messier 的時刻。這些描述，本來就不是學術例子，而是現場語言。

所以最好的測法，通常就是：

拿你最近那個真的很煩的 case。
拿那個你已經覺得 AI 一直在修表面的 case。
拿那個你自己都開始懷疑「我們是不是一開始就站錯樓層」的 case。

把 TXT 丟進去。
讓它先 route。
先讀主要家族、次要家族、broken invariant、first fix direction。
看它是不是把你從原本那條一直歪掉的路，拉回一條比較有紀律的起點。

這種測法最有意思的地方在於，你不是在看它答不答得出正確答案。
你是在看：

它有沒有讓你少修錯一次。

很多時候，這比「一次全對」更有實際價值。
因為真實世界的 debugging，本來就常常不是一次封神，而是少繞幾圈。

8.8 如果我拿去試，覺得它有些地方還不夠完整，怎麼辦

這個問題問得很好。
而且我會說，這其實正是 Atlas 現在最健康的使用態度。

因為 Atlas 從頭到尾都不是用「我已經全部完成」的姿態在出場。
主頁、Hub、Fixes 層、demo pack、Freeze Note 都在傳達一種很一致的氣質：

核心已經站住。
入口已經能用。
修復導向結構已經長出來。
但整張地圖還值得被更多真實案例壓。

所以如果你拿去試之後覺得：

某個邊界太模糊。
某個子家族其實還不夠穩。
某種 case 很容易讓 primary / secondary 互相搶戲。
某個 first fix direction 寫得還不夠明確。
某個 demo 還不夠有說服力。

這不是「你把它用壞了」。
這反而是 Atlas 最該接收的資訊。
因為一張 troubleshooting atlas，如果真的要長厚，本來就需要被真實 case 打。
你發現的不是麻煩，而是它繼續長肌肉的地方。

所以比較健康的想法不是：

「它不完美，所以算了。」

而是：

「它核心已經可用，所以我的 case 現在可以變成它下一步長厚的證據。」

這也是為什麼後面收尾那一章，我們會把「歡迎壓測、歡迎提 issue、歡迎真實 case 來打」當成重要態度。
因為這不是客套。
這本來就是 Atlas 變成熟的一部分。

8.9 所以，第八章真正想講的是什麼

如果把這一章壓成一句話，我會這樣寫：

Atlas 不是只給專家用的，也不是你要先全部學完才能碰的系統。
它比較像一張你可以先拿來找方向、先避免修錯樓層、再慢慢長理解的地圖。

這句話很重要。

因為很多好的系統，最後不是輸在理論不夠強。
而是輸在入口太像考試。
Atlas 現在的 public surface，尤其是 Router TXT 與 Usage Guide，剛好相反。它們其實很努力在做一件很不花俏、但很值錢的事：讓一個人不用先成為 Atlas 學家，也可以開始用 Atlas 的方式看問題。

而只要你開始這樣看問題，後面的七大家族、子家族、demo、repair-facing 結構，就不再像一堆硬詞。
它們會慢慢變成你手上那個 case 的地圖。

本章小結

這一章在講：

你不是工程師，也可以用。
你不用先背完整本 Atlas，也可以開始。
它不是全自動修復器，但它會幫你少走錯樓層。
如果兩個家族都很像，不要硬裝精準，先看邊界理由和證據夠不夠。
如果 demo 看不懂，先看人話版本也沒問題。
如果你有真實 case，反而最值得拿來試。

簡單講，Atlas 最健康的打開方式，不是把它當成神諭。
而是把它當成一張你可以先拿來校正方向的地圖。

Bonus 1

這七大母框架，當初到底是怎麼長出來的

如果你一路看到這裡，心裡大概會冒出一個很合理的問題：

好，所以這七大母框架，到底是怎麼來的。
是某天靈光一閃，還是我真的把一堆問題拆到最後，才慢慢把它們雕出來。

答案比較接近後者。
而且老實說，這條路一開始並沒有看起來那麼浪漫。
比較像一個人把很多零件、很多圖紙、很多半成品引擎攤在桌上，然後盯著它們看很久，最後發現：

「不對，這堆東西下面其實有共同母骨架，只是以前還沒有被完整雕出來。」

當初我先做出來的，不是 Atlas。
而是 **RAG 16 問題清單（RAG 16 Problem Map，RAG 16 問題地圖）**。
那一版其實已經很有用，因為只要你把文章發到對的地方，受眾就會非常精準。
做 RAG（檢索增強生成）的人，看了就知道你不是亂講。
卡在檢索、資料接地、chunk（切塊）、向量庫、排序、引用、回應穩定性的人，也會很快看懂這套東西是在解什麼。

但我那時候心裡一直有一個更大的不滿足。
不是因為那一版不好。剛好相反，是因為它太準了，所以受眾也太窄了。
你只要發到對的地方，它會打得很準。
可是對我來說，WFGY 的目標從來不是只做一份給 RAG 工程師看的 debugging list（除錯清單）。WFGY（萬法歸一）的目標，是想把底層邏輯重寫，去碰更大範圍的 reasoning system（推理系統）和 world-scale debugging surface（世界尺度除錯介面）。

所以那時候我做了一件很直白的事。
我把三代引擎擺在桌上，看著那份 RAG 16 問題清單，開始想：

如果我不要只停在「這 16 類問題很好用」，
而是要把它們背後共同的母結構雕出來，會長什麼樣子。

這裡有一件事我想講清楚。RAG 16 問題清單，本來就不是憑空冒出來的。
它其實和 **WFGY 1.0（第一代 WFGY）** 是同根同源。更準確地說，RAG 16 問題清單，本來就是從 **WFGY 1.0 引擎（第一代 WFGY 引擎）** 那套推理邏輯一路推演出來的 practical map（實用地圖）。只是那一版還比較集中在 RAG（檢索增強生成）這個落地場景，所以你看到的是一張很能打、但還比較偏一個子領域的問題圖。

接下來，我做的事就是升級。
我先拿 **WFGY 1.0（第一代 WFGY）** 加上 **RAG 16 問題清單（RAG 16 Problem Map，RAG 16 問題地圖）**，再透過更底層的**張力宇宙邏輯（Tension Universe Logic，張力宇宙底層邏輯）**去往上提。這個張力宇宙邏輯，不是目前 WFGY 已公開的產品面，它比較像一個還在更深層鍛造中的底盤，未來也可能會變成收費服務。

這裡我不展開講，因為這份 PDF 的主角不是它。
但它在當時扮演的角色很重要，它像一個高壓雕刻機，逼我不要只看 16 類問題表面長什麼樣，而是去看：

這 16 類問題背後，到底有哪些更高階的共同故障起點。

也就是在那一輪裡，七大母框架的輪廓開始慢慢浮出來。

先把母框架雕出來，還不算最難。
真正困難的，是下面的**子樹（Subtree，子樹）**。
因為母框架像是大地圖，子樹才是你開始真的能走路、真的能修、真的能被真實案例打磨的地方。而子樹不是那種你坐在椅子上靠直覺就能亂長出來的東西。
它必須經過更細的 case（案例）、更細的邊界衝突、更細的誤修（Misrepair，誤修）觀察，才會開始有資格長成穩定分支。
這一塊我當時也用了另外一套推理引擎輔助，但這裡我先不展開。
因為我想讓這份 PDF 保持清楚，而不是突然多開三個宇宙。

但這還不是最關鍵的一步。
真正把七大母框架站穩的，是後來的**邊界測試（Boundary Test，邊界測試）和壓力測試（Stress Test，壓力測試）**。

這裡就輪到 **WFGY 3.0（第三代 WFGY）** 登場。
你這輪給我的那份 TXT，很清楚地把自己定義成一個 **131 個 S 級問題包（131 S-class problems pack，131 個 S 級問題包）**，而且強調整個 pack（資料包）要嚴格停留在 **有效層（effective layer，有效層）**。它不是在宣稱自己已經解完這些世紀問題，而是在提供一種跨領域、可結構比較、可做實驗、可做框架壓力測試的 encoding surface（編碼介面）。它甚至明講，這個 pack 的一個核心用途，就是讓模型去看不同領域的問題，觀察是不是有同一組 structural skeleton（結構骨架）在重複出現。

這對我來說，非常重要。
因為如果七大母框架只是 RAG 世界裡看起來合理，那它還不夠。
我需要知道的是：

當我把它丟進更高張力、跨數學、物理、AI、意識、治理、文明風險這些不同領域的問題裡，它還站不站得住。

而 **WFGY 3.0（第三代 WFGY）** 這 131 題，剛好就是最好的壓力室。
它不是一兩題漂亮範例。它是一整個 **S-class pack（S 級問題包）**。
它要求的是跨領域結構觀察、統一骨架比較、重複出現的 state space（狀態空

間)、observables(可觀測量)、tension functional(張力函數)、counterfactual worlds(反事實世界)這些東西是不是能站得住。它甚至把這件事寫進 GO 模式裡，要模型去看不同問題之間是不是共享同一組 skeleton(骨架)，而不是各領域各講各的。

所以你現在看到的 Atlas，不是「我先決定要有七類，再把故事補上去」。
比較像是：

先有 WFGY 1.0(第一代 WFGY)和 RAG 16 問題清單(RAG 16 Problem Map，RAG 16 問題地圖)這條根。再用更底層的張力宇宙邏輯把母框架往上雕。
子樹部分再靠額外推理結構慢慢長。最後，拿 WFGY 3.0(第三代 WFGY)這 131 題 S 級問題去做壓力測試，逼自己回答一個很殘酷的問題：

這個母框架，是真的有跨場景穩定性。
還是只是剛好在某個小領域裡好看而已。

老實說，做到這裡，我才開始覺得 Atlas 真的站起來了。
不是因為它變得很大。而是因為它開始承受得住撞擊。

所以如果你要我用一句話總結 Bonus 1，我會這樣講：

七大母框架不是設計出來的裝飾品。
它比較像是從 RAG 16 問題地圖一路往上提煉，再被更高張力問題反覆壓測後，慢慢被逼出來的母骨架。

Bonus 2

如果都知道了錯誤，那能不能把錯誤變成一種預防勝於治療的方法

做到 Atlas 這一步之後，我心裡自然又冒出下一個問題。
而且這個問題，其實比 Atlas 更危險，也更有趣。

那就是：

如果我們都已經知道 AI 常見的錯誤地圖了，
那我們有沒有辦法，不只是拿它來事後分類，
而是反過來把它變成一種「生成前治理（Pre-generative Governance，生成前治理）」的方法。

這就是我現在正在做的下一步。
我暫時把它叫做：

反向 Atlas（Inverse Atlas，反向 Atlas）

Atlas 比較像事後分流。
你先讓系統說話、先讓問題長出來，再回頭看它到底是哪裡先壞。
反向 Atlas 則是在想一件更狠的事：

如果錯誤模式已經知道了，那我能不能在它開口之前，就先審核它有沒有資格開口。

Generation is not a default right（生成不是預設權利）。
Generation is an authorized act（生成是一種被授權的行為）。

我很喜歡這種寫法。
因為它不是在說「AI 很壞，所以要關起來」。
它是在說：

現在很多 AI 的錯誤，不是因為它不會說。
而是因為它太容易說，而且說得太早、太滿、太細、太有自信。

所以反向 Atlas 的核心不是再多加一層回答技巧。
它的核心是把**回答權（authorization to answer，回答授權）**這件事，從默認開放，改成需要通過結構審核。

Inverse Atlas 的任務也寫得非常清楚，它要求在 substantive output（實質輸出）之前，先做 pre-generative governance（生成前治理），而且直接明定幾個偏執到很可愛的原則：

偏向合法的不完整（lawful incompleteness，合法的不完整），不要非法的完整（illegal completeness，非法的完整）偏向明確限制（explicit limits，明確限制），不要假的確定性（fake certainty，假的確定性）
偏向結構適配(structural fit，結構適配)，不要主題相似(topic resemblance，主題相似)

這其實就是把 Atlas 的思路反過來。
以前是：

你先講了，我再來判你錯在哪。

現在變成：

你想講之前，先證明你有合法的理由、合法的解析度、合法的邊界分離、合法的 public emission ceiling（公開輸出上限）。

這裡最讓我覺得有意思的，是它把很多現在 AI 最常見的毛病，直接升級成治理條文。例如：

不要把 topic resemblance（主題相似）當權威，structural fit（結構適配）才有權威。

不要因為語氣強就假裝解析度高，high resolution（高解析度）不是風格，而是一種 authorization state（授權狀態）。

如果 neighboring-cut separation（鄰近切口分離）不夠，就不准升解析度。
如果 broken invariant（壞掉的規則）根本沒碰到，那 repair（修復）就是 cosmetic（美容式）而不是 structural（結構式）。

如果 evidence（證據）太薄，就讓 uncertainty（不確定性）老老實實留著。
safe stop（安全停止）是合法成功，不是失敗。

說真的，這套東西有一種很「權力回收」的味道。
而且我覺得這很對。因為現在很多 AI 幻覺（hallucination，幻覺）、過度自信、亂修、亂補、亂升解析度的問題，背後其實有一個很大的共同根源：

它的回答權力太大了。

它常常沒有先經過 problem legitimacy（問題合法性）、world alignment（世界對齊）、resolution sovereignty（解析度主權）、public emission sovereignty（公開輸出主權）的審核，就直接開始長篇大論。
Inverse Atlas 很狠地把這件事改掉。
它要求每次輸出都要先回答：

這個問題現在真的有資格被高解析度回答嗎。
這個 primary route（主要路由）真的比 nearest competing route（最近競爭

路由）更強嗎。
這個 repair（修復）真的碰到 broken invariant（壞掉規則）了嗎。
這個公開輸出，會不會超過現在的 legitimacy ceiling（合法性天花板） 。

如果沒有，就降級。
再沒有，就停。而且它還真的把這些狀態做成四種 mode（模式）：

STOP（停止）
COARSE（粗顆粒）
UNRESOLVED（未解決）
AUTHORIZED（已授權）

這套東西最讓我有感的地方，就是它把「預防勝於治療」這件事，從一種口號，慢慢變成一套可以部署的結構。

Atlas 是說：

你已經開始說錯了，來，我們看看你錯在哪。

Inverse Atlas（反向 Atlas）則是在說：

等一下。你要不要先證明你現在有資格說這麼多。

我自己覺得，效果其實還不錯。
而且這可能是一條很新的路。
因為它不是在拼命讓 AI 說更多。
它是在做一件現在非常少人願意做的事：

把 AI 的權力收回來。

不是讓它越來越像一個沒有審核的萬能發言人。
而是讓它回到一個更健康的位置：

先審。
再放行。
沒有過，就不要硬講。
證據薄，就不要假裝高解析度。
鄰近切口還沒分開，就不要升級成 node-level certainty（節點層確定性）。
修復碰不到 broken invariant（壞掉規則），就不要裝成 structural repair（結構修復）。

這其實就是我現在想做的下一步方法論。
簡單講就是：

從事後分類，走向生成前治理。

前者是 Atlas。
後者是 Inverse Atlas（反向 Atlas）。

而且我覺得，這條路未來會很重要。
因為你只要看現在 AI 最常見的災難，你就會發現，很多問題不是它完全沒有能力，而是它在**不該高解析度時給了高解析度，不該自信時給了自信，不該放行時直接放行**。
所以如果我們真的想把幻覺(hallucination，幻覺)、誤修(misrepair，誤修)、假精準（fake precision，假精準）這些東西壓下來，光靠事後補救其實不夠。
有一部分工作，必須拉到生成前。

這，就是反向 Atlas（Inverse Atlas，反向 Atlas）現在正在走的方向。
這沒有要賣你們新產品，反向 Altas 還是免費的，還是 MIT 授權

最後，真的謝謝你看到這裡

如果你一路看到這裡，那先謝謝你。
因為這份文件雖然是透過 AI 協助完成，但裡面的很多雕琢、很多骨架、很多取捨、很多「這裡到底該不該講這麼重、那裡到底要不要再收一下」的細節，其實花了我不少時間。

我知道現在網路上內容很多。
大家也都很忙。所以如果你最後願意把這份文件看到這裡，對我來說，真的已經很感謝了。

如果你覺得這份內容有幫助，
不管是 Atlas（故障地圖）這一塊，
還是後面這個 Inverse Atlas（反向 Atlas）的方法方向，
都很歡迎你到 GitHub 上面給我一個星星。
這對我來說不是什麼虛榮數字，它比較像是一種很直接的鼓勵，告訴我：

這條路有人看見。
而且它值得繼續做下去。

如果你願意，也歡迎持續關注我，或者到我們的 Discord（Discord 社群）一起討論。接下來我們很快就會開始推進 **反向 Atlas**（Inverse Atlas，**反向 Atlas**）這套新方法論。

它不是單純的 prompt 技巧，也不是傳統那種「先生成，再補救」的套路。
它更像是一套**生成前治理（Pre-generative Governance，生成前治理）**的思路：

在 AI 開口之前，先做審核。
沒有授權，就不要放行。
沒有碰到結構，就不要裝成修好了。
沒有足夠證據，就不要硬給高解析度答案。

簡單講，
我們現在發現，很多 AI 的幻覺（hallucination，幻覺）和錯誤，不是因為它完全不會，而是因為它權力太大了。
我們的做法，就是把這個權力收回來。
要先經過一套更嚴格的審核，才允許它開始說話。

而如果這條路真的走通，
那它可能不是在幫 AI 長更多嘴巴。
而是在幫 AI 長出一個比較像真正成熟系統的喉嚨。

就先到這裡。
感謝你一路看到最後。

附錄 A

研究與新聞來源

這一份附錄的角色，不是拿來堆砌權威感。
它的真正作用，是讓讀者知道，這份文件前面提到的幾個關鍵觀察，背後不是只有個人體感，而是有公開研究、產業調查與新聞報導作為支撐。

你可以把這一頁理解成：
不是在說「我一定完全正確」，而是在說「我有做功課，而且我願意把參考依據攤出來」。

A.1 AI 開發採用率與使用現況

1. Stack Overflow Developer Survey 2025

主題：AI 工具在開發流程中的採用率、使用頻率、信任度與痛點。

用途：支撐第一章中「AI coding（AI 程式生成）已經不是未來式，而是很多開發者的日常工具」這個觀察。

連結：

https://survey.stackoverflow.co/2025/ai

2. GitHub Survey / Research on AI Coding Tools

主題：開發者在工作中使用 AI coding tools（AI 程式編碼工具）的普及程度與行為觀察。

用途：補強「AI 已經進入真實工作流」這件事，不只是小圈圈實驗。

連結：

https://github.blog/news-insights/research/survey-ai-wave-grows/

A.2 AI 對開發速度的影響

3. Microsoft Research / The Impact of AI on Developer Productivity: Evidence from GitHub Copilot

主題：GitHub Copilot（GitHub 副駕駛）在特定程式任務上的速度提升研究。

用途：支撐第一章中「AI 在某些 coding task（程式任務）上確實能加速」的描述。

連結：

https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/

4. GitHub Copilot 官方產品頁

主題：Copilot（副駕駛）如何利用目前編輯區、鄰近檔案與工作區上下文進行建議生成。

用途：支撐第一章中「AI 可能因上下文濃度與鄰近訊號而優先抓到表面症狀」的解釋。

連結：

https://github.com/features/copilot

A.3 真實專案中的摩擦與驗證成本

5. METR / Early 2025 AI Study on Experienced Open Source Developers

主題：資深開源開發者在熟悉 codebase（程式碼庫）中使用 AI 工具時的實際效率研究。

用途：支撐第一章中「在真實專案與熟悉 repo（倉庫）中，AI 並不一定自動等於更快」的觀察。

連結：

https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

6. Reuters 對 METR 研究的新聞整理

主題：AI 工具在真實開發情境下，可能因驗證與修正成本而使整體時間增加。

用途：作為大眾媒體角度的補充來源，方便一般讀者理解研究結論。

連結：

如果要保守，你可以不放 Reuters，直接只放 METR 主文；如果要放，建議最後排版時再確認你要用哪一篇新聞版本。

A.4 關於 AI 輸出信任與「幾乎對但差一點」的痛點

7. Stack Overflow Developer Survey 2025 / AI Section

主題：開發者對 AI 產出準確性的信任度，以及「answers are almost right, but not quite（答案幾乎對，但差一點）」的常見挫折。

用途：支撐第一章中「真正折磨人的，不是 AI 完全不會，而是它太像會」這個核心描述。

連結：

https://survey.stackoverflow.co/2025/ai

A.5 補充說明

以上來源主要用於支撐本文件第一章的幾個基本觀察：

- AI coding（AI 程式生成）採用率持續升高
- 在某些任務上，AI 確實能加速開發
- 真實工作流中的驗證、排錯、與 first fix（第一修法）選錯，仍然是非常大的成本來源
- 使用者真正痛苦的，常常不是 AI 完全不會，而是它很像會，卻把你帶進一條從第一步就切錯的修法路徑

附錄 B

重要 repo 入口總表

這一份附錄的用途，是讓讀者在看完 PDF 之後，可以直接回到 repo 裡對照原始資料。

我建議這頁在排版時做成「文字連結清單」，不要塞太多花樣，越清楚越好。

B.1 WFGY 主入口

WFGY 主頁（WFGY Repository Home）

整個系統的主入口，包含 Atlas（故障地圖）、Problem Map（問題地圖）、core（核心）、Recognition（認可紀錄）等對外導航。

連結：

https://github.com/onestardao/WFGY

B.2 Atlas 主線文件

Atlas 主頁（Problem Map 3.0 Troubleshooting Atlas）

這份 PDF 的主軸來源，也是目前 Atlas 對外最重要的總覽頁。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/wfgy-ai-problem-map-troubleshooting-atlas.md

Atlas Hub（Atlas 內部導航頁）

如果想更系統地理解 Atlas 文件夾裡的結構，這一頁最適合當目錄。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/README.md

Atlas Final Freeze（Atlas 正式凍結骨架）

七大母框架（Seven-family Mother Table，七大家族母表）、高價值子樹（High-value Subtree，高價值子樹）、標準節點層（Canonical Node Layer，標準節點層）的核心來源。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/atlas-final-freeze-v1.md

---

B.3 Router 與上手入口

Router TXT（路由 TXT 主檔）

目前最適合直接丟進大型語言模型（LLM，大型語言模型）開始試用的 compact pack（精簡包）。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/troubleshooting-atlas-router-v1.txt

Router Usage Guide（Router 使用說明）

最短上手流程與 output contract（輸出契約）閱讀順序的主要來源。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/troubleshooting-atlas-router-v1-usage.md

Router Freeze Note（Router 凍結說明）

用來理解 Router v1（第一版 Router）目前的產品邊界、核心承諾與非承諾範圍。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/troubleshooting-atlas-router-v1-freeze-note.md

---

B.4 修復與 demo

Fixes Hub（修復導向延伸層）

Atlas diagnosis（地圖診斷）之後，往 first repair direction（第一修法方向）延伸的重要橋樑。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/Fixes/README.md

Official Demos（官方 demo）

目前四個官方 MVP demos（最小可行驗證示範）的總入口。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/Atlas/Fixes/official/demos/README.md

---

B.5 問題地圖與核心脈絡

Problem Map 主頁（ProblemMap README）

如果想理解較早期的 Problem Map（問題地圖）系統與對外脈絡，可以從這一頁開始。

連結：

https://github.com/onestardao/WFGY/blob/main/ProblemMap/README.md

WFGY Core（WFGY 核心頁）

理解 WFGY 2.0（第二代 WFGY）與更深層引擎脈絡的重要入口。

連結：

https://github.com/onestardao/WFGY/blob/main/core/README.md

WFGY 3.0 / Event Horizon（WFGY 3.0 / Event Horizon）

131 個 S 級問題（131 S-class problems，131 個 S 級問題）與更高張力壓力測試（Stress Test，壓力測試）脈絡的公開入口。

連結：

https://github.com/onestardao/WFGY/blob/main/TensionUniverse/EventHorizon/README.md

---

B.6 Recognition 與外部足跡

Recognition Map（Recognition README，認可地圖）

整理 WFGY / Problem Map（問題地圖）被外部專案、文件、清單或研究注意到的紀錄。

連結：

https://github.com/onestardao/WFGY/blob/main/recognition/README.md

---

附錄 C

代表性子家族簡表

這一份附錄不是完整樹狀圖。

它的任務只有一個：

讓讀者知道，七大家族下面已經不是空白。

而且子家族（Subfamily，子家族）與子樹（Subtree，子樹）不是亂切，而是已經有可以研究、可以使用、也可以繼續被壓測的初步結構。

下面這份清單，我刻意只放「代表性」例子，不追求完整百科全書。

C.1 接地與證據完整性

Grounding & Evidence Integrity（接地與證據完整性）

檢索錨點漂移（Retrieval Anchor Drift）

檢索到的資料表面相關，但真正該抓的錨點漂掉了。

語義接地錯位（Semantic Grounding Mismatch）

不是沒資料，而是資料與目標語義沒有真的接上。

合成真值接地（Synthetic Truth Grounding）

系統在合成內容時，看起來很順，但其實缺乏可靠的 truth anchor（真值錨點）。

分布外世界接地失敗（OOD World Grounding Failure）

在訓練世界裡看似成立的東西，一進真實世界就接不上。

C.2 推理與推進完整性

Reasoning & Progression Integrity（推理與推進完整性）

推進鏈斷橋（Progression Bridge Failure）

中間該存在的推進橋樑被跳過。

中途偷步（Mid-chain Shortcutting）

表面很流暢，但中間必要步驟被壓扁。

因果方向誤導（Causal Direction Distortion）

結果看起來合理，但因果方向其實走反了。

C.3 狀態與連續性完整性

State & Continuity Integrity（狀態與連續性完整性）

記憶連續性斷裂（Memory Continuity Failure）

前後內容明明相關，系統卻像換了一顆腦袋。

角色污染（Role Contamination）

不同任務、不同角色、不同責任邊界互相滲透，造成判斷漂移。

所有權持續性破裂（Ownership Continuity Break）

前後狀態不是沒有資訊，而是資訊不再屬於同一條線。

多代理連續性不穩（Multi-agent Continuity Instability）

多代理（Multi-agent，多代理）之間的狀態傳承開始鬆動。

C.4 執行與契約完整性

Execution & Contract Integrity（執行與契約完整性）

啟動排序失敗（Bootstrap Ordering Failure）

流程不是不會跑，而是起跑順序就已經錯了。

部署死鎖（Deployment Deadlock）

不同段落彼此等待，最後誰都沒真的往前。

未準備好就執行（Pre-Readiness Execution Failure）

理論上開始了，實際上 readiness（準備度）還沒滿。

橋接完整性失敗（Bridge Integrity Failure）

步驟和步驟之間的橋沒有接好。

C.5 觀測與可診斷性完整性

Observability & Diagnosability Integrity（觀測與可診斷性完整性）

故障路徑不透明（Failure Path Opacity）

系統不是沒有壞，而是你看不到它怎麼壞。

監督覆蓋失敗（Oversight Coverage Failure）

該觀察、該監看、該能回頭查的地方，根本沒有被照到。

一致性可觀測缺口（Coherence Observability Deficit）

你看到輸出，但看不到一致性怎麼一步步掉下去。

C.6 邊界與安全完整性

Boundary & Safety Integrity（邊界與安全完整性）

停止條件鬆脫（Stop-condition Weakening）

本來該停的地方，系統開始裝作沒看見。

風險邊界漂移（Risk Boundary Drift）

該保守的時候不保守，該限制的地方開始鬆掉。

授權錯位（Authorization Mismatch）

系統在沒有足夠合法性的情況下，輸出了過高解析度的回答。

C.7 表徵與定位完整性

Representation & Localization Integrity（表徵與定位完整性）

OCR / 版面結構失敗（OCR / Layout Structural Failure）

不是內容沒有，而是版面與載體先把內容弄壞了。

表徵載體扭曲（Representation Carrier Distortion）

裝內容的盒子歪了，所以裡面的東西看起來也跟著歪。

語義定位失敗（Semantic Localization Failure）

資訊不是缺，而是被放錯位置、對錯槽位。

錨點與容器錯配（Anchor-Container Mismatch）

內容想接在某個錨點上，但承載它的容器不對。

C.8 附錄 C 的閱讀方式

這一頁不是叫你把每個子家族（Subfamily，子家族）全部背起來。
它只是要讓你看懂一件事：

七大家族下面已經開始有比較細的結構。
而這些結構存在的目的，不是把地圖畫得更花，而是讓第一修法少一點誤修（Misrepair，誤修）。

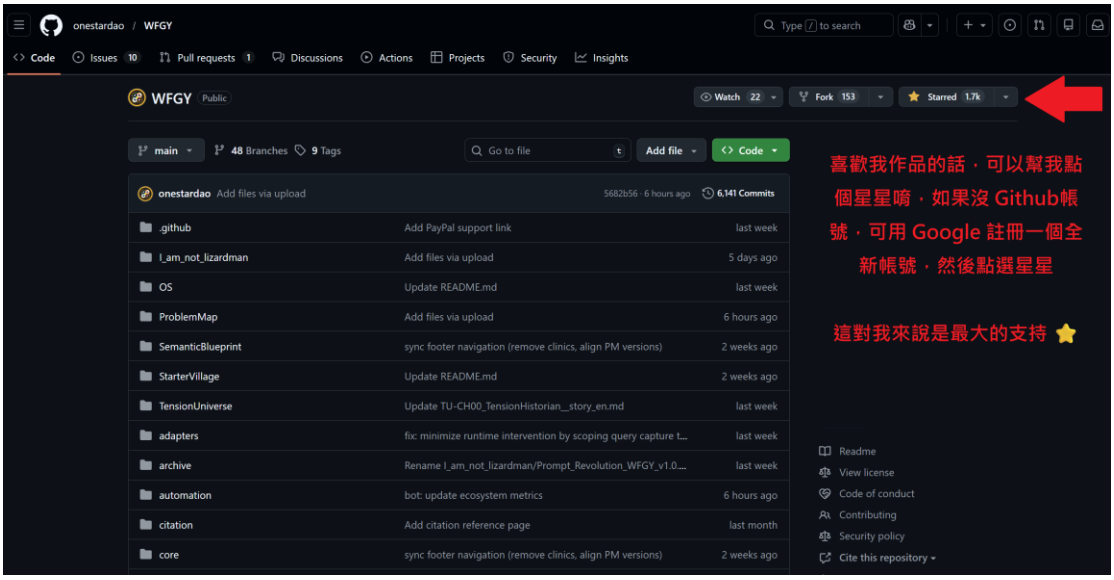
如果你後續真的要深入研究，完整更操作化的子家族訊號，目前在 Router TXT（Router TXT，路由 TXT 主檔）裡會更明顯。

阿紫 BigBig 的感謝與連結

如果你看到這裡，先真心謝謝你。

這份文件雖然是透過 AI 協助整理與撰寫，
但很多骨架、很多取捨、很多細節、很多哪裡該收、哪裡該放、哪裡要更白話、哪裡要更嚴謹的雕琢，其實花了我不少時間。

如果你覺得這份內容有幫助，
不管是 Atlas（故障地圖）這一塊，
還是後面提到的 Inverse Atlas（反向 Atlas，反向 Atlas）方向，
都很歡迎你到 GitHub 給我一個星星。
對我來說，這不只是數字，
它更像是一種很直接的鼓勵，告訴我這條路值得繼續做下去。



GitHub

WFGY GitHub：
https://github.com/onestardao/WFGY

WFGY Discord 社群

歡迎加入我們的 Discord（Discord 社群），一起討論 Atlas（故障地圖）、AI Debug（AI 除錯）、WFGY（WFGY 系統）、與後續的 Inverse Atlas（反向 Atlas，反向 Atlas）方向：

https://discord.gg/KRxBsr6GYx

阿紫 BigBig 推薦課程：

商盟 AI 學院：輕鬆掌握生成式 AI，邁向數位未來（line 群，我也在裡面唷）

Line（Line 聯絡入口）：

https://line.me/R/ti/p/@726xsmrl